

Bus Bridge Server

commit-327ab86

Generated by Doxygen 1.9.8

1 Bus Bridge Server	1
2 Test List	3
3 Hierarchical Index	5
3.1 Class Hierarchy	5
4 Class Index	7
4.1 Class List	7
5 File Index	9
5.1 File List	9
6 Class Documentation	11
6.1 BaseSlave Class Reference	11
6.1.1 Detailed Description	12
6.1.2 Constructor & Destructor Documentation	12
6.1.2.1 ~BaseSlave()	12
6.1.3 Member Function Documentation	12
6.1.3.1 getData()	12
6.1.3.2 getId()	13
6.1.3.3 getStatus()	13
6.1.3.4 setData()	13
6.1.3.5 start()	13
6.1.3.6 stop()	13
6.2 BusServer Class Reference	14
6.2.1 Detailed Description	14
6.2.2 Constructor & Destructor Documentation	15
6.2.2.1 BusServer()	15
6.2.3 Member Function Documentation	15
6.2.3.1 getSlaveManager()	15
6.2.3.2 listen()	15
6.2.3.3 send()	15
6.2.3.4 setup()	16
6.2.3.5 start()	16
6.2.4 Member Data Documentation	16
6.2.4.1 buffer	16
6.2.4.2 listening_address	17
6.2.4.3 listening_fd	17
6.2.4.4 slave_manager	17
6.2.4.5 wemos_bridge_connected	17
6.3 CO2Slave Class Reference	17
6.3.1 Detailed Description	19
6.3.2 Constructor & Destructor Documentation	20

6.3.2.1 CO2Slave()	20
6.3.3 Member Function Documentation	20
6.3.3.1 getData()	20
6.3.3.2 getId()	20
6.3.3.3 getStatus()	20
6.3.3.4 setData()	20
6.3.3.5 start()	21
6.3.3.6 stop()	21
6.3.4 Member Data Documentation	21
6.3.4.1 command	21
6.3.4.2 fd	21
6.3.4.3 i2c_address	21
6.3.4.4 id	21
6.3.4.5 power_state	21
6.3.4.6 state_packet	22
6.4 FanSlave Class Reference	22
6.4.1 Detailed Description	23
6.4.2 Constructor & Destructor Documentation	24
6.4.2.1 FanSlave()	24
6.4.3 Member Function Documentation	24
6.4.3.1 getData()	24
6.4.3.2 getId()	24
6.4.3.3 getStatus()	24
6.4.3.4 setData()	24
6.4.3.5 start()	25
6.4.3.6 stop()	25
6.4.4 Member Data Documentation	25
6.4.4.1 current_speed	25
6.4.4.2 fd	25
6.4.4.3 i2c_address	25
6.4.4.4 id	25
6.4.4.5 state_packet	26
6.5 LightSlave Class Reference	26
6.5.1 Detailed Description	27
6.5.2 Constructor & Destructor Documentation	28
6.5.2.1 LightSlave()	28
6.5.3 Member Function Documentation	28
6.5.3.1 getData()	28
6.5.3.2 getId()	28
6.5.3.3 getStatus()	28
6.5.3.4 setData()	28
6.5.3.5 start()	29

6.5.3.6 stop()	29
6.5.4 Member Data Documentation	29
6.5.4.1 color_state	29
6.5.4.2 fd	29
6.5.4.3 i2c_address	29
6.5.4.4 id	29
6.5.4.5 state_packet	30
6.6 RGBData Struct Reference	30
6.6.1 Detailed Description	30
6.6.2 Member Data Documentation	31
6.6.2.1 B	31
6.6.2.2 G	31
6.6.2.3 R	31
6.7 RGBSlave Class Reference	31
6.7.1 Detailed Description	33
6.7.2 Constructor & Destructor Documentation	34
6.7.2.1 RGBSlave()	34
6.7.3 Member Function Documentation	34
6.7.3.1 getData()	34
6.7.3.2 getId()	34
6.7.3.3 getStatus()	34
6.7.3.4 setData()	34
6.7.3.5 start()	35
6.7.3.6 stop()	35
6.7.4 Member Data Documentation	35
6.7.4.1 color_state	35
6.7.4.2 fd	35
6.7.4.3 i2c_address	35
6.7.4.4 id	35
6.7.4.5 state_packet	36
6.8 sensor_data Union Reference	36
6.8.1 Detailed Description	36
6.8.2 Member Data Documentation	37
6.8.2.1 co2	37
6.8.2.2 generic	37
6.8.2.3 heartbeat	37
6.8.2.4 humidity	37
6.8.2.5 light	37
6.8.2.6 rgb_light	37
6.8.2.7 temperature	37
6.9 sensor_packet::sensor_data Union Reference	38
6.9.1 Detailed Description	38

6.9.2 Member Data Documentation	38
6.9.2.1 co2	38
6.9.2.2 generic	38
6.9.2.3 heartbeat	39
6.9.2.4 humidity	39
6.9.2.5 light	39
6.9.2.6 rgb_light	39
6.9.2.7 temperature	39
6.10 sensor_header Struct Reference	40
6.10.1 Detailed Description	40
6.10.2 Member Data Documentation	40
6.10.2.1 length	40
6.10.2.2 ptype	41
6.11 sensor_heartbeat Struct Reference	41
6.11.1 Detailed Description	42
6.11.2 Member Data Documentation	42
6.11.2.1 metadata	42
6.12 sensor_metadata Struct Reference	42
6.12.1 Detailed Description	43
6.12.2 Member Data Documentation	43
6.12.2.1 sensor_id	43
6.12.2.2 sensor_type	43
6.13 sensor_packet Struct Reference	43
6.13.1 Detailed Description	44
6.13.2 Member Data Documentation	45
6.13.2.1 data	45
6.13.2.2 header	45
6.14 sensor_packet_co2 Struct Reference	45
6.14.1 Detailed Description	46
6.14.2 Member Data Documentation	46
6.14.2.1 metadata	46
6.14.2.2 value	46
6.15 sensor_packet_fan Struct Reference	47
6.15.1 Detailed Description	47
6.15.2 Member Data Documentation	48
6.15.2.1 metadata	48
6.15.2.2 speed	48
6.16 sensor_packet_generic Struct Reference	48
6.16.1 Detailed Description	49
6.16.2 Member Data Documentation	49
6.16.2.1 metadata	49
6.17 sensor_packet_humidity Struct Reference	49

6.17.1 Detailed Description	50
6.17.2 Member Data Documentation	50
6.17.2.1 metadata	50
6.17.2.2 value	50
6.18 sensor_packet_light Struct Reference	51
6.18.1 Detailed Description	51
6.18.2 Member Data Documentation	52
6.18.2.1 metadata	52
6.18.2.2 target_state	52
6.19 sensor_packet_rgb_light Struct Reference	52
6.19.1 Detailed Description	53
6.19.2 Member Data Documentation	53
6.19.2.1 blue_state	53
6.19.2.2 green_state	53
6.19.2.3 metadata	53
6.19.2.4 red_state	54
6.20 sensor_packet_temperature Struct Reference	54
6.20.1 Detailed Description	55
6.20.2 Member Data Documentation	55
6.20.2.1 metadata	55
6.20.2.2 value	55
6.21 SlaveManager Class Reference	55
6.21.1 Detailed Description	56
6.21.2 Constructor & Destructor Documentation	56
6.21.2.1 SlaveManager()	56
6.21.3 Member Function Documentation	56
6.21.3.1 createSlave()	56
6.21.3.2 deleteSlave()	56
6.21.3.3 getSlave()	57
6.21.3.4 initialize()	57
6.21.3.5 ledControl()	57
6.21.4 Member Data Documentation	57
6.21.4.1 address	57
6.21.4.2 i2c_fd	58
6.21.4.3 initialized	58
6.21.4.4 slaves	58
6.22 TemperatureData Struct Reference	58
6.22.1 Detailed Description	59
6.22.2 Member Data Documentation	59
6.22.2.1 Temp	59
6.23 TemperatureSlave Class Reference	59
6.23.1 Detailed Description	62

6.23.2 Constructor & Destructor Documentation	62
6.23.2.1 TemperatureSlave()	62
6.23.3 Member Function Documentation	62
6.23.3.1 getData()	62
6.23.3.2 getId()	62
6.23.3.3 getStatus()	62
6.23.3.4 setData()	62
6.23.3.5 start()	63
6.23.3.6 stop()	63
6.23.4 Member Data Documentation	63
6.23.4.1 fd	63
6.23.4.2 i2c_address	63
6.23.4.3 id	63
6.23.4.4 power_state	63
6.23.4.5 temperature	63
7 File Documentation	65
7.1 include/BaseSlave.h File Reference	65
7.2 BaseSlave.h	66
7.3 include/BusServer.h File Reference	66
7.3.1 Macro Definition Documentation	67
7.3.1.1 BUFFER_SIZE	67
7.4 BusServer.h	67
7.5 include/CO2Slave.h File Reference	68
7.6 CO2Slave.h	68
7.7 include/FanSlave.h File Reference	69
7.7.1 Typedef Documentation	70
7.7.1.1 FanData	70
7.8 FanSlave.h	70
7.9 include/LightSlave.h File Reference	71
7.9.1 Typedef Documentation	72
7.9.1.1 LightData	72
7.10 LightSlave.h	72
7.11 include/math.h File Reference	72
7.11.1 Detailed Description	73
7.11.2 Function Documentation	73
7.11.2.1 add()	73
7.11.2.2 subtract()	73
7.12 math.h	74
7.13 include/packets.h File Reference	74
7.13.1 Detailed Description	76
7.13.2 Enumeration Type Documentation	76

7.13.2.1 PacketType	76
7.13.2.2 SensorType	77
7.13.3 Function Documentation	77
7.13.3.1 __attribute__()	77
7.13.4 Variable Documentation	77
7.13.4.1 blue_state	77
7.13.4.2 data	78
7.13.4.3 green_state	78
7.13.4.4 header	78
7.13.4.5 length	78
7.13.4.6 metadata	78
7.13.4.7 ptype	78
7.13.4.8 red_state	78
7.13.4.9 sensor_id	79
7.13.4.10 sensor_type	79
7.13.4.11 speed	79
7.13.4.12 target_state	79
7.13.4.13 value	79
7.14 packets.h	80
7.15 include/RGBSlave.h File Reference	81
7.15.1 Function Documentation	82
7.15.1.1 __attribute__()	82
7.15.2 Variable Documentation	82
7.15.2.1 __attribute__	82
7.15.2.2 B	82
7.15.2.3 G	82
7.15.2.4 R	82
7.16 RGBSlave.h	83
7.17 include/SlaveManager.h File Reference	83
7.18 SlaveManager.h	84
7.19 include/TemperatureSlave.h File Reference	85
7.19.1 Function Documentation	86
7.19.1.1 __attribute__()	86
7.19.2 Variable Documentation	86
7.19.2.1 __attribute__	86
7.19.2.2 Temp	86
7.20 TemperatureSlave.h	86
7.21 README.md File Reference	87
7.22 src/BaseSlave.cpp File Reference	87
7.23 BaseSlave.cpp	87
7.24 src/BusServer.cpp File Reference	87
7.24.1 Macro Definition Documentation	88

7.24.1.1 RGB_SLAVE_ADDRESS	88
7.25 BusServer.cpp	88
7.26 src/CO2Slave.cpp File Reference	90
7.27 CO2Slave.cpp	91
7.28 src/FanSlave.cpp File Reference	92
7.29 FanSlave.cpp	92
7.30 src/LightSlave.cpp File Reference	92
7.31 LightSlave.cpp	93
7.32 src/main.cpp File Reference	94
7.32.1 Function Documentation	94
7.32.1.1 main()	94
7.33 main.cpp	94
7.34 src/math.cpp File Reference	95
7.34.1 Detailed Description	95
7.34.2 Function Documentation	95
7.34.2.1 add()	95
7.34.2.2 subtract()	96
7.35 math.cpp	96
7.36 src/RGBSlave.cpp File Reference	96
7.37 RGBSlave.cpp	97
7.38 src/SlaveManager.cpp File Reference	98
7.38.1 Macro Definition Documentation	98
7.38.1.1 MASTER_ADDRESS	98
7.39 SlaveManager.cpp	99
7.40 src/TemperatureSlave.cpp File Reference	100
7.41 TemperatureSlave.cpp	100
7.42 tests/test_math.cpp File Reference	101
7.42.1 Detailed Description	101
7.42.2 Function Documentation	102
7.42.2.1 TEST() [1/3]	102
7.42.2.2 TEST() [2/3]	102
7.42.2.3 TEST() [3/3]	102
7.43 test_math.cpp	102
Index	103

Chapter 1

Bus Bridge Server

Chapter 2

Test List

File [test_math.cpp](#)

MathTest.Add

- Verifies that the `add` function correctly computes the sum of two integers.
- Example: `add(2, 3)` should return 5.

MathTest.Subtract

- Verifies that the `subtract` function correctly computes the difference between two integers.
- Examples:
 - `subtract(10, 3)` should return 7.
 - `subtract(9, 3)` should return 6.

MathTest.SubtractNegative

- Verifies that the `subtract` function handles subtraction with negative integers correctly.
- Example: `subtract(10, -3)` should return 13.

Chapter 3

Hierarchical Index

3.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

BaseSlave	11
CO2Slave	17
FanSlave	22
LightSlave	26
RGBSlave	31
TemperatureSlave	59
BusServer	14
RGBData	30
sensor_data	36
sensor_packet::sensor_data	38
sensor_header	40
sensor_heartbeat	41
sensor_metadata	42
sensor_packet	43
sensor_packet_co2	45
sensor_packet_fan	47
sensor_packet_generic	48
sensor_packet_humidity	49
sensor_packet_light	51
sensor_packet_rgb_light	52
sensor_packet_temperature	54
SlaveManager	55
TemperatureData	58

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

BaseSlave	11
BusServer	14
CO2Slave	17
FanSlave	22
LightSlave	26
RGBData	30
RGBSlave	31
sensor_data	
36	
sensor_packet::sensor_data	
38	
sensor_header	
Header structure for sensor packets	40
sensor_heartbeat	
Structure for heartbeat packets	41
sensor_metadata	
Structure for sensor metadata, which is always included in any packet	42
sensor_packet	
Union structure for the entire sensor packet	43
sensor_packet_co2	
Structure for CO2 sensor packets	45
sensor_packet_fan	
Structure for packets intended to control the fan speed	47
sensor_packet_generic	
Structure for generic sensor packets	48
sensor_packet_humidity	
Structure for humidity sensor packets	49
sensor_packet_light	
Structure for light sensor packets	51
sensor_packet_rgb_light	
Structure for RGB light sensor packets	52
sensor_packet_temperature	
Structure for temperature sensor packets	54
SlaveManager	55
TemperatureData	58
TemperatureSlave	59

Chapter 5

File Index

5.1 File List

Here is a list of all files with brief descriptions:

include/BaseSlave.h	65
include/BusServer.h	66
include/CO2Slave.h	68
include/FanSlave.h	69
include/LightSlave.h	71
include/math.h	
Header file for math.cpp	72
include/packets.h	
Header file for packets.h	74
include/RGBSlave.h	81
include/SlaveManager.h	83
include/TemperatureSlave.h	85
src/BaseSlave.cpp	87
src/BusServer.cpp	87
src/CO2Slave.cpp	90
src/FanSlave.cpp	92
src/LightSlave.cpp	92
src/main.cpp	94
src/math.cpp	
Implementation of basic math operations	95
src/RGBSlave.cpp	96
src/SlaveManager.cpp	98
src/TemperatureSlave.cpp	100
tests/test_math.cpp	
Unit tests for mathematical operations using Google Test framework	101

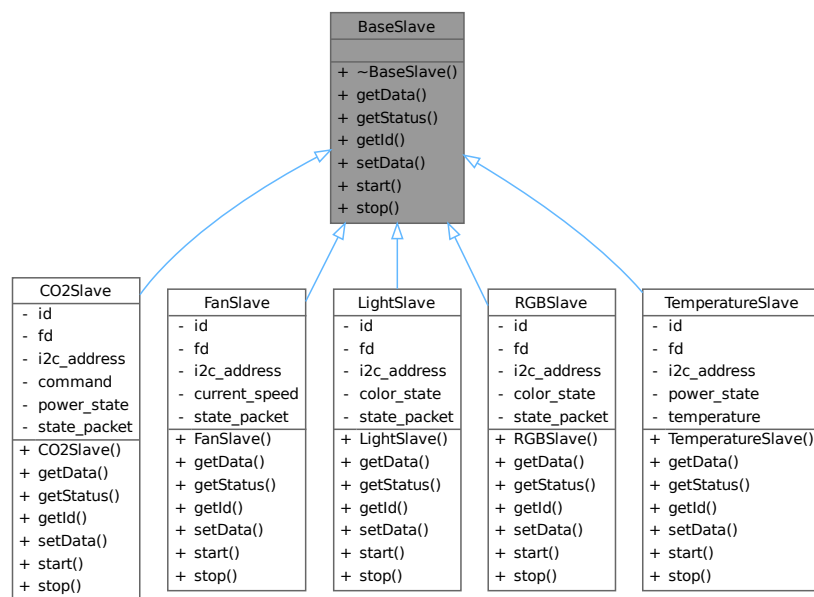
Chapter 6

Class Documentation

6.1 BaseSlave Class Reference

```
#include <BaseSlave.h>
```

Inheritance diagram for BaseSlave:



Collaboration diagram for BaseSlave:

BaseSlave
+ ~BaseSlave() + getData() + getStatus() + getId() + setData() + start() + stop()

Public Member Functions

- virtual [~BaseSlave](#) ()=default
- virtual const void * [getData](#) ()=0
- virtual bool [getStatus](#) ()=0
- virtual int [getId](#) ()=0
- virtual void [setData](#) (void *[data](#))=0
- virtual void [start](#) (int i2c_fd)=0
- virtual void [stop](#) ()=0

6.1.1 Detailed Description

Definition at line 6 of file [BaseSlave.h](#).

6.1.2 Constructor & Destructor Documentation

6.1.2.1 ~BaseSlave()

```
virtual BaseSlave::~~BaseSlave ( ) [virtual], [default]
```

6.1.3 Member Function Documentation

6.1.3.1 getData()

```
virtual const void * BaseSlave::getData ( ) [pure virtual]
```

Implemented in [FanSlave](#), [LightSlave](#), [RGBSlave](#), [TemperatureSlave](#), and [CO2Slave](#).

6.1.3.2 getId()

```
virtual int BaseSlave::getId ( ) [pure virtual]
```

Implemented in [FanSlave](#), [LightSlave](#), [RGBSlave](#), [TemperatureSlave](#), and [CO2Slave](#).

6.1.3.3 getStatus()

```
virtual bool BaseSlave::getStatus ( ) [pure virtual]
```

Implemented in [FanSlave](#), [LightSlave](#), [RGBSlave](#), [TemperatureSlave](#), and [CO2Slave](#).

6.1.3.4 setData()

```
virtual void BaseSlave::setData (
    void * data ) [pure virtual]
```

Implemented in [FanSlave](#), [LightSlave](#), [RGBSlave](#), [TemperatureSlave](#), and [CO2Slave](#).

6.1.3.5 start()

```
virtual void BaseSlave::start (
    int i2c_fd ) [pure virtual]
```

Implemented in [FanSlave](#), [LightSlave](#), [RGBSlave](#), [TemperatureSlave](#), and [CO2Slave](#).

6.1.3.6 stop()

```
virtual void BaseSlave::stop ( ) [pure virtual]
```

Implemented in [FanSlave](#), [LightSlave](#), [RGBSlave](#), [TemperatureSlave](#), and [CO2Slave](#).

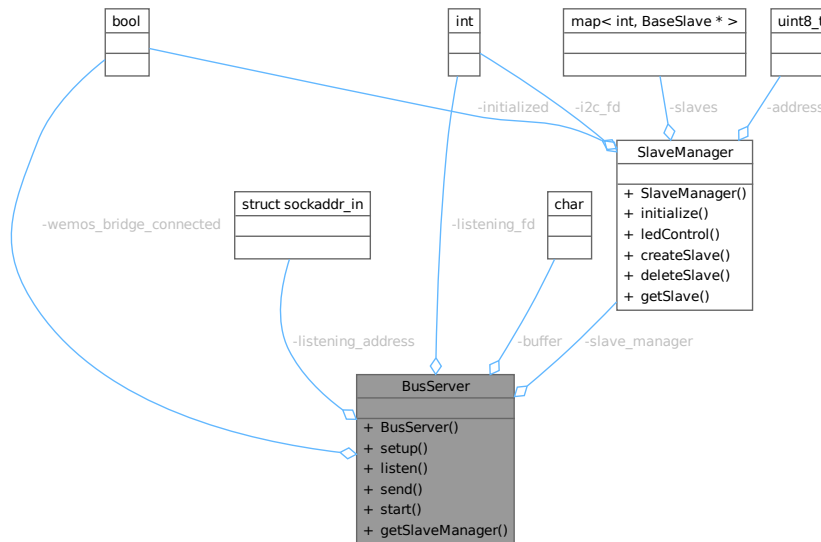
The documentation for this class was generated from the following file:

- [include/BaseSlave.h](#)

6.2 BusServer Class Reference

```
#include <BusServer.h>
```

Collaboration diagram for BusServer:



Public Member Functions

- `BusServer ()`
- void `setup (std::string ip, int port)`
Setup for the IP socket and the I2C slaves connection.
- void `listen ()`
Open the underlying socket for incoming connections.
- void `send (struct sensor_packet *pkt, int fd)`
Send a sensor packet to a connected network client.
- void `start ()`
Start the main loop of the `BusServer`.
- `SlaveManager` & `getSlaveManager ()`
balls

Private Attributes

- int `listening_fd`
- struct `sockaddr_in` `listening_address`
- bool `wemos_bridge_connected` = false
- char `buffer [BUFFER_SIZE]`
- `SlaveManager` `slave_manager`

6.2.1 Detailed Description

Definition at line 12 of file `BusServer.h`.

6.2.2 Constructor & Destructor Documentation

6.2.2.1 BusServer()

```
BusServer::BusServer ( ) [inline]
```

Definition at line 14 of file [BusServer.h](#).

6.2.3 Member Function Documentation

6.2.3.1 getSlaveManager()

```
SlaveManager & BusServer::getSlaveManager ( )
```

balls

Definition at line 179 of file [BusServer.cpp](#).

6.2.3.2 listen()

```
void BusServer::listen ( )
```

Open the underlying socket for incoming connections.

Exceptions

<code>std::runtime_error</code>	if the listening fails
---------------------------------	------------------------

Definition at line 59 of file [BusServer.cpp](#).

6.2.3.3 send()

```
void BusServer::send (
    struct sensor_packet * pkt,
    int fd )
```

Send a sensor packet to a connected network client.

Depending on the data set in the packet header, send a certain amount of data to the client.

Parameters

<i>pkt</i>	A pointer to the sensor_packet struct to send over
<i>fd</i>	The file descriptor to send the packet over

Exceptions

<code>std::runtime_error</code>	if the sending fails for any reason
---------------------------------	-------------------------------------

Definition at line 66 of file [BusServer.cpp](#).

6.2.3.4 setup()

```
void BusServer::setup (
    std::string ip,
    int port )
```

Setup for the IP socket and the I2C slaves connection.

This method will set up a socket for listening on the network and will also tell the underlying [SlaveManager](#) object to initialize its I2C bus

Parameters

<i>ip</i>	The IP address to listen on within the network
<i>port</i>	The TCP port to listen on

Exceptions

<code>std::invalid_argument</code>	if the passed IP address or port number are invalid
<code>std::runtime_error</code>	if the socket creation fails

Definition at line 19 of file [BusServer.cpp](#).

6.2.3.5 start()

```
void BusServer::start ( )
```

Start the main loop of the [BusServer](#).

This will first initialize the underlying I2C connections to the directly-connected slave devices, and then start accepting and processing network clients

Definition at line 76 of file [BusServer.cpp](#).

6.2.4 Member Data Documentation**6.2.4.1 buffer**

```
char BusServer::buffer[BUFFER_SIZE] [private]
```

Definition at line 60 of file [BusServer.h](#).

6.2.4.2 listening_address

```
struct sockaddr_in BusServer::listening_address [private]
```

Definition at line 58 of file [BusServer.h](#).

6.2.4.3 listening_fd

```
int BusServer::listening_fd [private]
```

Definition at line 56 of file [BusServer.h](#).

6.2.4.4 slave_manager

```
SlaveManager BusServer::slave_manager [private]
```

Definition at line 62 of file [BusServer.h](#).

6.2.4.5 wemos_bridge_connected

```
bool BusServer::wemos_bridge_connected = false [private]
```

Definition at line 59 of file [BusServer.h](#).

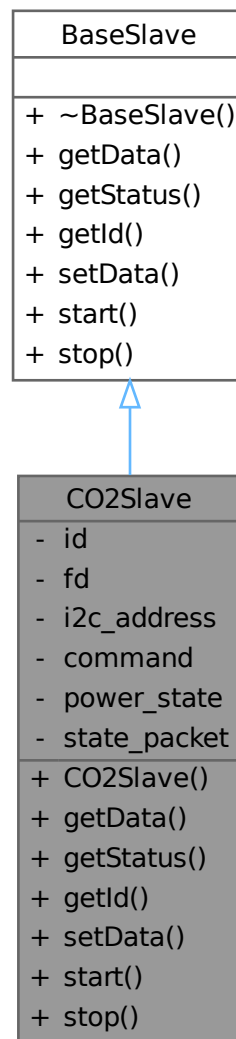
The documentation for this class was generated from the following files:

- [include/BusServer.h](#)
- [src/BusServer.cpp](#)

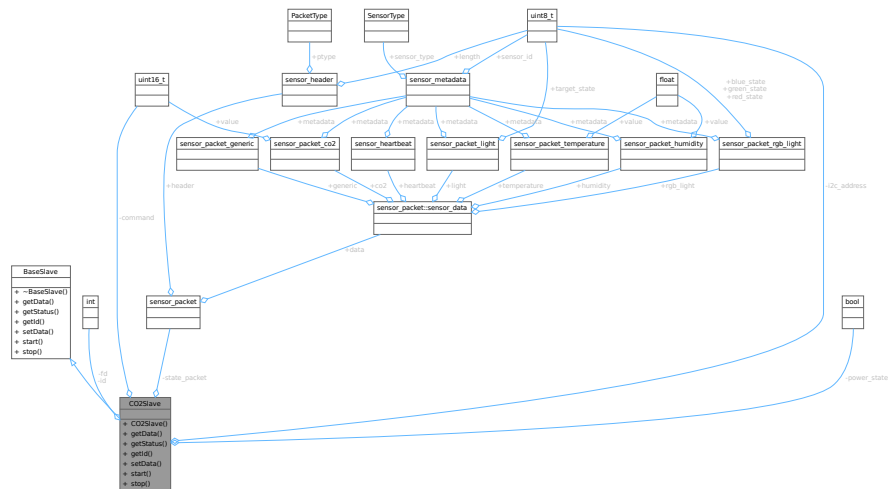
6.3 CO2Slave Class Reference

```
#include <CO2Slave.h>
```

Inheritance diagram for CO2Slave:



Collaboration diagram for CO2Slave:



Public Member Functions

- `CO2Slave (uint8_t id, uint8_t i2c_address)`
- `const void * getData ()` override
- `bool getStatus ()` override
- `int getId ()` override
- `void setData (void *data)` override
- `void start (int i2c_fd)` override
- `void stop ()` override

Public Member Functions inherited from BaseSlave

- virtual ~BaseSlave ()=default

Private Attributes

- int **id**
- int **fd**
- uint8_t **i2c_address**
- uint16_t **command**
- bool **power_state** = true
- struct **sensor_packet** **state_packet**

6.3.1 Detailed Description

Definition at line 7 of file CO2Slave.h.

6.3.2 Constructor & Destructor Documentation

6.3.2.1 CO2Slave()

```
CO2Slave::CO2Slave (
    uint8_t id,
    uint8_t i2c_address )
```

Definition at line 5 of file [CO2Slave.cpp](#).

6.3.3 Member Function Documentation

6.3.3.1 getData()

```
const void * CO2Slave::getData ( ) [override], [virtual]
```

Implements [BaseSlave](#).

Definition at line 18 of file [CO2Slave.cpp](#).

6.3.3.2 getId()

```
int CO2Slave::getId ( ) [override], [virtual]
```

Implements [BaseSlave](#).

Definition at line 33 of file [CO2Slave.cpp](#).

6.3.3.3 getStatus()

```
bool CO2Slave::getStatus ( ) [override], [virtual]
```

Implements [BaseSlave](#).

Definition at line 31 of file [CO2Slave.cpp](#).

6.3.3.4 setData()

```
void CO2Slave::setData (
    void * data ) [override], [virtual]
```

Implements [BaseSlave](#).

Definition at line 35 of file [CO2Slave.cpp](#).

6.3.3.5 start()

```
void CO2Slave::start (
    int i2c_fd ) [override], [virtual]
```

Implements [BaseSlave](#).

Definition at line 37 of file [CO2Slave.cpp](#).

6.3.3.6 stop()

```
void CO2Slave::stop ( ) [override], [virtual]
```

Implements [BaseSlave](#).

Definition at line 39 of file [CO2Slave.cpp](#).

6.3.4 Member Data Documentation

6.3.4.1 command

```
uint16_t CO2Slave::command [private]
```

Definition at line 22 of file [CO2Slave.h](#).

6.3.4.2 fd

```
int CO2Slave::fd [private]
```

Definition at line 20 of file [CO2Slave.h](#).

6.3.4.3 i2c_address

```
uint8_t CO2Slave::i2c_address [private]
```

Definition at line 21 of file [CO2Slave.h](#).

6.3.4.4 id

```
int CO2Slave::id [private]
```

Definition at line 19 of file [CO2Slave.h](#).

6.3.4.5 power_state

```
bool CO2Slave::power_state = true [private]
```

Definition at line 24 of file [CO2Slave.h](#).

6.3.4.6 state_packet

```
struct sensor\_packet CO2Slave::state_packet [private]
```

Definition at line 25 of file [CO2Slave.h](#).

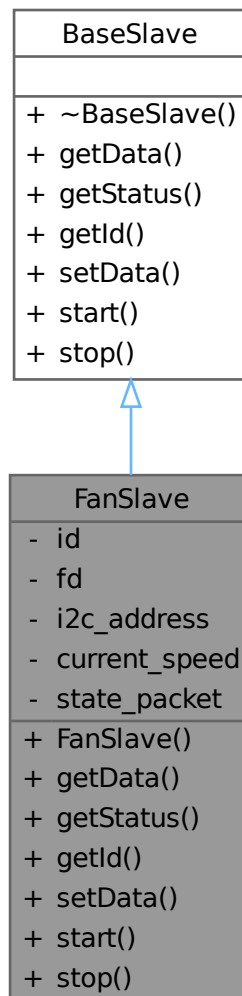
The documentation for this class was generated from the following files:

- [include/CO2Slave.h](#)
- [src/CO2Slave.cpp](#)

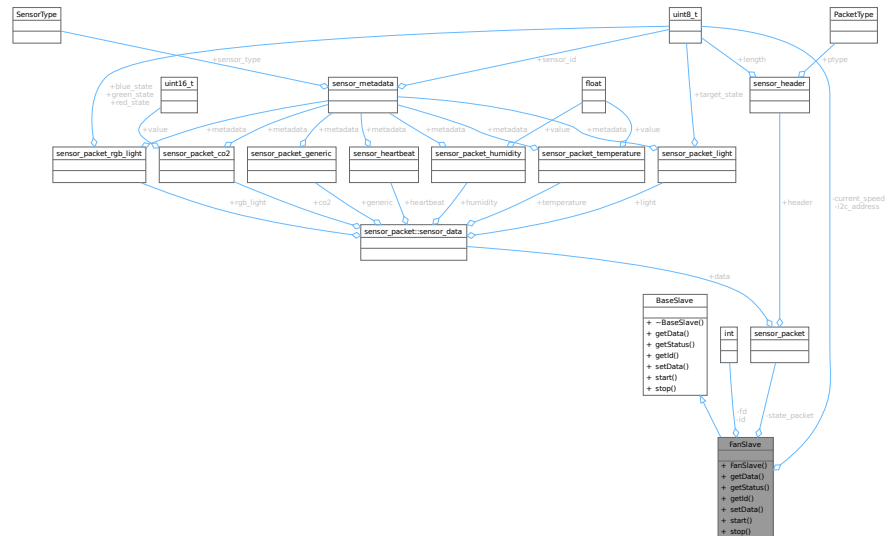
6.4 FanSlave Class Reference

```
#include <FanSlave.h>
```

Inheritance diagram for FanSlave:



Collaboration diagram for FanSlave:



Public Member Functions

- [FanSlave](#) (uint8_t id, uint8_t i2c_address)
- const void * [getData](#) ()
- bool [getStatus](#) ()
- int [getId](#) ()
- void [setData](#) (void *data)
- void [start](#) (int i2c_fd)
- void [stop](#) ()

Public Member Functions inherited from [BaseSlave](#)

- virtual [~BaseSlave](#) ()=default

Private Attributes

- int id
- int fd
- uint8_t i2c_address
- [FanData](#) current_speed
- struct [sensor_packet](#) state_packet

6.4.1 Detailed Description

Definition at line 10 of file [FanSlave.h](#).

6.4.2 Constructor & Destructor Documentation

6.4.2.1 FanSlave()

```
FanSlave::FanSlave (
    uint8_t id,
    uint8_t i2c_address )
```

Definition at line 3 of file [FanSlave.cpp](#).

6.4.3 Member Function Documentation

6.4.3.1 getData()

```
const void * FanSlave::getData ( ) [virtual]
```

Implements [BaseSlave](#).

Definition at line 5 of file [FanSlave.cpp](#).

6.4.3.2 getId()

```
int FanSlave::getId ( ) [virtual]
```

Implements [BaseSlave](#).

Definition at line 15 of file [FanSlave.cpp](#).

6.4.3.3 getStatus()

```
bool FanSlave::getStatus ( ) [virtual]
```

Implements [BaseSlave](#).

Definition at line 7 of file [FanSlave.cpp](#).

6.4.3.4 setData()

```
void FanSlave::setData (
    void * data ) [virtual]
```

Implements [BaseSlave](#).

Definition at line 17 of file [FanSlave.cpp](#).

6.4.3.5 start()

```
void FanSlave::start (
    int i2c_fd ) [virtual]
```

Implements [BaseSlave](#).

Definition at line 23 of file [FanSlave.cpp](#).

6.4.3.6 stop()

```
void FanSlave::stop ( ) [virtual]
```

Implements [BaseSlave](#).

Definition at line 25 of file [FanSlave.cpp](#).

6.4.4 Member Data Documentation

6.4.4.1 current_speed

```
FanData FanSlave::current_speed [private]
```

Definition at line 25 of file [FanSlave.h](#).

6.4.4.2 fd

```
int FanSlave::fd [private]
```

Definition at line 22 of file [FanSlave.h](#).

6.4.4.3 i2c_address

```
uint8_t FanSlave::i2c_address [private]
```

Definition at line 23 of file [FanSlave.h](#).

6.4.4.4 id

```
int FanSlave::id [private]
```

Definition at line 21 of file [FanSlave.h](#).

6.4.4.5 state_packet

```
struct sensor\_packet FanSlave::state_packet [private]
```

Definition at line 27 of file [FanSlave.h](#).

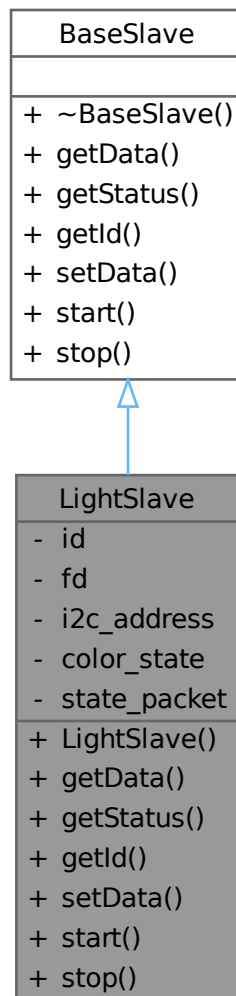
The documentation for this class was generated from the following files:

- include/[FanSlave.h](#)
- src/[FanSlave.cpp](#)

6.5 LightSlave Class Reference

```
#include <LightSlave.h>
```

Inheritance diagram for LightSlave:



6.5.2 Constructor & Destructor Documentation

6.5.2.1 LightSlave()

```
LightSlave::LightSlave (
    uint8_t id,
    uint8_t i2c_address )
```

Definition at line 7 of file [LightSlave.cpp](#).

6.5.3 Member Function Documentation

6.5.3.1 getData()

```
const void * LightSlave::getData ( ) [virtual]
```

Implements [BaseSlave](#).

Definition at line 15 of file [LightSlave.cpp](#).

6.5.3.2 getId()

```
int LightSlave::getId ( ) [virtual]
```

Implements [BaseSlave](#).

Definition at line 28 of file [LightSlave.cpp](#).

6.5.3.3 getStatus()

```
bool LightSlave::getStatus ( ) [virtual]
```

Implements [BaseSlave](#).

Definition at line 23 of file [LightSlave.cpp](#).

6.5.3.4 setData()

```
void LightSlave::setData (
    void * data ) [virtual]
```

Implements [BaseSlave](#).

Definition at line 30 of file [LightSlave.cpp](#).

6.5.3.5 start()

```
void LightSlave::start (
    int i2c_fd ) [virtual]
```

Implements [BaseSlave](#).

Definition at line 39 of file [LightSlave.cpp](#).

6.5.3.6 stop()

```
void LightSlave::stop ( ) [virtual]
```

Implements [BaseSlave](#).

Definition at line 41 of file [LightSlave.cpp](#).

6.5.4 Member Data Documentation

6.5.4.1 color_state

```
LightData LightSlave::color_state [private]
```

Definition at line 24 of file [LightSlave.h](#).

6.5.4.2 fd

```
int LightSlave::fd [private]
```

Definition at line 21 of file [LightSlave.h](#).

6.5.4.3 i2c_address

```
uint8_t LightSlave::i2c_address [private]
```

Definition at line 22 of file [LightSlave.h](#).

6.5.4.4 id

```
int LightSlave::id [private]
```

Definition at line 20 of file [LightSlave.h](#).

6.5.4.5 state_packet

```
struct sensor\_packet LightSlave::state_packet [private]
```

Definition at line 26 of file [LightSlave.h](#).

The documentation for this class was generated from the following files:

- include/[LightSlave.h](#)
- src/[LightSlave.cpp](#)

6.6 RGBData Struct Reference

```
#include <RGBSlave.h>
```

Collaboration diagram for RGBData:



Public Attributes

- `uint8_t` [R](#)
- `uint8_t` [G](#)
- `uint8_t` [B](#)

6.6.1 Detailed Description

Definition at line 7 of file [RGBSlave.h](#).

6.6.2 Member Data Documentation

6.6.2.1 B

```
uint8_t RGBData::B
```

Definition at line 8 of file [RGBSlave.h](#).

6.6.2.2 G

```
uint8_t RGBData::G
```

Definition at line 8 of file [RGBSlave.h](#).

6.6.2.3 R

```
uint8_t RGBData::R
```

Definition at line 8 of file [RGBSlave.h](#).

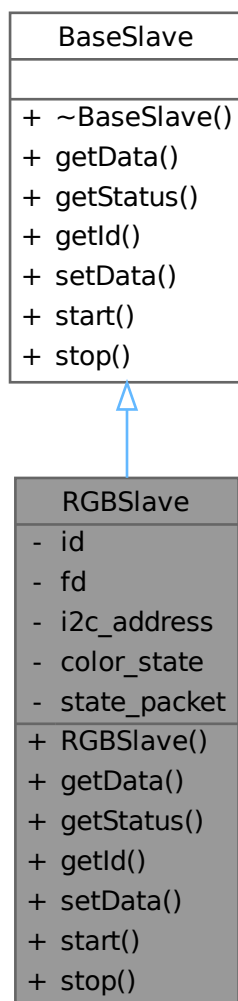
The documentation for this struct was generated from the following file:

- include/[RGBSlave.h](#)

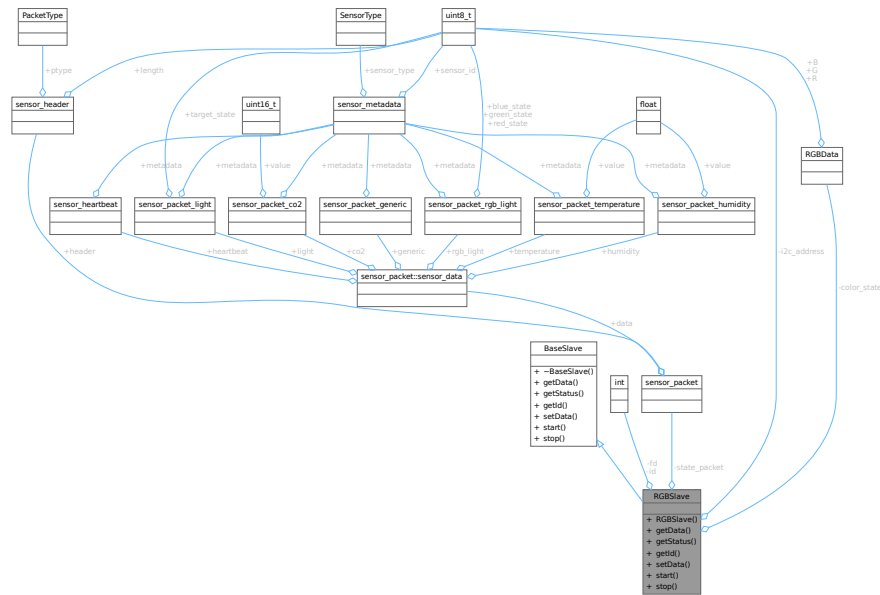
6.7 RGBSlave Class Reference

```
#include <RGBSlave.h>
```

Inheritance diagram for RGBSlave:



Collaboration diagram for RGBSlave:



Public Member Functions

- [RGBSlave](#) (uint8_t id, uint8_t i2c_address)
- const void * [getData](#) ()
- bool [getStatus](#) ()
- int [getId](#) ()
- void [setData](#) (void *data)
- void [start](#) (int i2c_fd)
- void [stop](#) ()

Public Member Functions inherited from [BaseSlave](#)

- virtual [~BaseSlave](#) ()=default

Private Attributes

- int [id](#)
- int [fd](#)
- uint8_t [i2c_address](#)
- [RGBData](#) [color_state](#)
- struct [sensor_packet](#) [state_packet](#)

6.7.1 Detailed Description

Definition at line 11 of file [RGBSlave.h](#).

6.7.2 Constructor & Destructor Documentation

6.7.2.1 RGBSlave()

```
RGBSlave::RGBSlave (
    uint8_t id,
    uint8_t i2c_address )
```

Definition at line 7 of file [RGBSlave.cpp](#).

6.7.3 Member Function Documentation

6.7.3.1 getData()

```
const void * RGBSlave::getData ( ) [virtual]
```

Implements [BaseSlave](#).

Definition at line 15 of file [RGBSlave.cpp](#).

6.7.3.2 getId()

```
int RGBSlave::getId ( ) [virtual]
```

Implements [BaseSlave](#).

Definition at line 42 of file [RGBSlave.cpp](#).

6.7.3.3 getStatus()

```
bool RGBSlave::getStatus ( ) [virtual]
```

Implements [BaseSlave](#).

Definition at line 33 of file [RGBSlave.cpp](#).

6.7.3.4 setData()

```
void RGBSlave::setData (
    void * data ) [virtual]
```

Implements [BaseSlave](#).

Definition at line 44 of file [RGBSlave.cpp](#).

6.7.3.5 start()

```
void RGBSlave::start (
    int i2c_fd ) [virtual]
```

Implements [BaseSlave](#).

Definition at line 65 of file [RGBSlave.cpp](#).

6.7.3.6 stop()

```
void RGBSlave::stop ( ) [virtual]
```

Implements [BaseSlave](#).

Definition at line 67 of file [RGBSlave.cpp](#).

6.7.4 Member Data Documentation

6.7.4.1 color_state

```
RGBData RGBSlave::color_state [private]
```

Definition at line 26 of file [RGBSlave.h](#).

6.7.4.2 fd

```
int RGBSlave::fd [private]
```

Definition at line 23 of file [RGBSlave.h](#).

6.7.4.3 i2c_address

```
uint8_t RGBSlave::i2c_address [private]
```

Definition at line 24 of file [RGBSlave.h](#).

6.7.4.4 id

```
int RGBSlave::id [private]
```

Definition at line 22 of file [RGBSlave.h](#).

6.7.4.5 state_packet

```
struct sensor_packet RGBSlave::state_packet [private]
```

Definition at line 28 of file [RGBSlave.h](#).

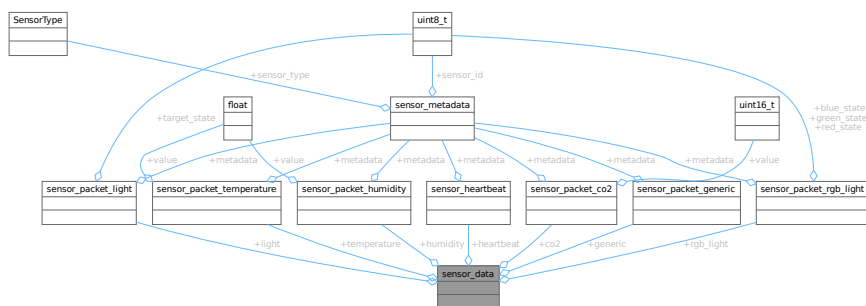
The documentation for this class was generated from the following files:

- include/[RGBSlave.h](#)
- src/[RGBSlave.cpp](#)

6.8 sensor_data Union Reference

```
#include <packets.h>
```

Collaboration diagram for `sensor_data`:



Public Attributes

- struct [sensor_heartbeat](#) heartbeat
- struct [sensor_packet_generic](#) generic
- struct [sensor_packet_temperature](#) temperature
- struct [sensor_packet_co2](#) co2
- struct [sensor_packet_humidity](#) humidity
- struct [sensor_packet_light](#) light
- struct [sensor_packet_rgb_light](#) rgb_light

6.8.1 Detailed Description

Definition at line 4 of file [packets.h](#).

6.8.2 Member Data Documentation

6.8.2.1 co2

```
struct sensor\_packet\_co2 sensor_data::co2
```

Definition at line 8 of file [packets.h](#).

6.8.2.2 generic

```
struct sensor\_packet\_generic sensor_data::generic
```

Definition at line 6 of file [packets.h](#).

6.8.2.3 heartbeat

```
struct sensor\_heartbeat sensor_data::heartbeat
```

Definition at line 5 of file [packets.h](#).

6.8.2.4 humidity

```
struct sensor\_packet\_humidity sensor_data::humidity
```

Definition at line 9 of file [packets.h](#).

6.8.2.5 light

```
struct sensor\_packet\_light sensor_data::light
```

Definition at line 10 of file [packets.h](#).

6.8.2.6 rgb_light

```
struct sensor\_packet\_rgb\_light sensor_data::rgb_light
```

Definition at line 11 of file [packets.h](#).

6.8.2.7 temperature

```
struct sensor\_packet\_temperature sensor_data::temperature
```

Definition at line 7 of file [packets.h](#).

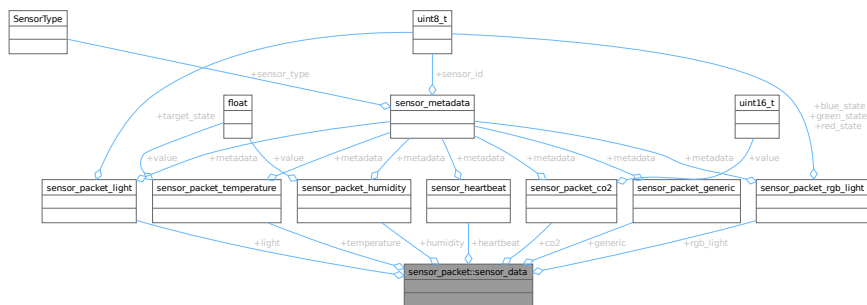
The documentation for this union was generated from the following file:

- [include/packets.h](#)

6.9 sensor_packet::sensor_data Union Reference

```
#include <packets.h>
```

Collaboration diagram for sensor_packet::sensor_data:



Public Attributes

- struct [sensor_heartbeat](#) heartbeat
- struct [sensor_packet_generic](#) generic
- struct [sensor_packet_temperature](#) temperature
- struct [sensor_packet_co2](#) co2
- struct [sensor_packet_humidity](#) humidity
- struct [sensor_packet_light](#) light
- struct [sensor_packet_rgb_light](#) rgb_light

6.9.1 Detailed Description

Definition at line 240 of file [packets.h](#).

6.9.2 Member Data Documentation

6.9.2.1 co2

```
struct sensor\_packet\_co2 sensor_packet::sensor_data::co2
```

Definition at line 244 of file [packets.h](#).

6.9.2.2 generic

```
struct sensor\_packet\_generic sensor_packet::sensor_data::generic
```

Definition at line 242 of file [packets.h](#).

6.9.2.3 heartbeat

```
struct sensor_heartbeat sensor_packet::sensor_data::heartbeat
```

Definition at line 241 of file [packets.h](#).

6.9.2.4 humidity

```
struct sensor_packet_humidity sensor_packet::sensor_data::humidity
```

Definition at line 245 of file [packets.h](#).

6.9.2.5 light

```
struct sensor_packet_light sensor_packet::sensor_data::light
```

Definition at line 246 of file [packets.h](#).

6.9.2.6 rgb_light

```
struct sensor_packet_rgb_light sensor_packet::sensor_data::rgb_light
```

Definition at line 247 of file [packets.h](#).

6.9.2.7 temperature

```
struct sensor_packet_temperature sensor_packet::sensor_data::temperature
```

Definition at line 243 of file [packets.h](#).

The documentation for this union was generated from the following file:

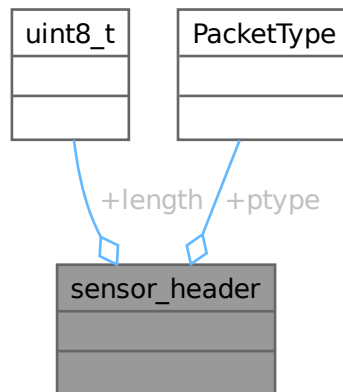
- [include/packets.h](#)

6.10 sensor_header Struct Reference

Header structure for sensor packets.

```
#include <packets.h>
```

Collaboration diagram for sensor_header:



Public Attributes

- [uint8_t length](#)
Length of the packet excluding the header.
- [PacketType ptype](#)
Type of the packet as PacketType (DATA, HEARTBEAT, etc.).

6.10.1 Detailed Description

Header structure for sensor packets.

Definition at line 41 of file [packets.h](#).

6.10.2 Member Data Documentation

6.10.2.1 length

```
uint8_t sensor_header::length
```

Length of the packet excluding the header.

Definition at line 43 of file [packets.h](#).

6.10.2.2 ptype

`PacketType sensor_header::ptype`

Type of the packet as PacketType (DATA, HEARTBEAT, etc.).

Definition at line 45 of file [packets.h](#).

The documentation for this struct was generated from the following file:

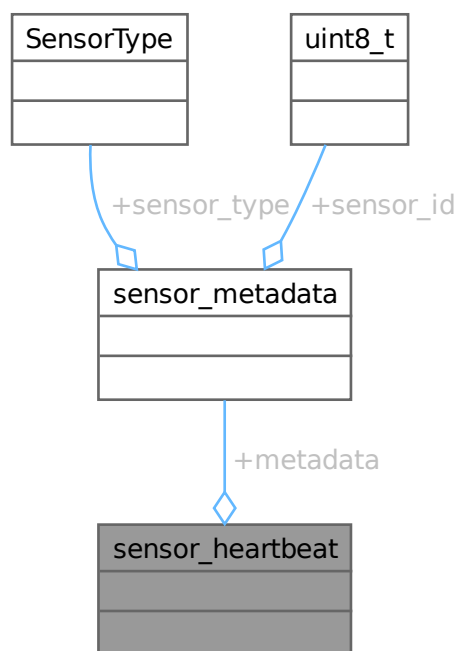
- [include/packets.h](#)

6.11 sensor_heartbeat Struct Reference

Structure for heartbeat packets.

```
#include <packets.h>
```

Collaboration diagram for sensor_heartbeat:



Public Attributes

- struct [sensor_metadata metadata](#)

6.11.1 Detailed Description

Structure for heartbeat packets.

This structure contains the type and ID of the sensor being addressed. This structure is used for heartbeat packets sent by the sensors to indicate they are still alive.

Definition at line 70 of file [packets.h](#).

6.11.2 Member Data Documentation

6.11.2.1 metadata

```
struct sensor\_metadata sensor_heartbeat::metadata
```

Definition at line 71 of file [packets.h](#).

The documentation for this struct was generated from the following file:

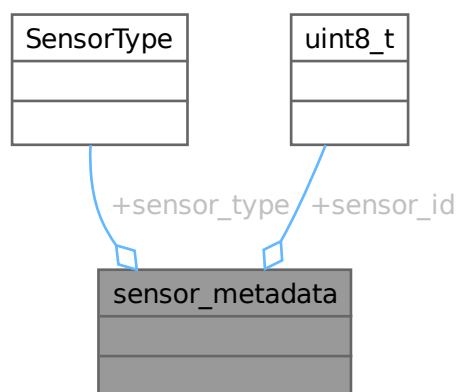
- [include/packets.h](#)

6.12 [sensor_metadata](#) Struct Reference

Structure for sensor metadata, which is always included in any packet.

```
#include <packets.h>
```

Collaboration diagram for [sensor_metadata](#):



Public Attributes

- **SensorType sensor_type**
Type of the sensor being addressed as SensorType (one byte)
- **uint8_t sensor_id**
ID of the sensor being addressed.

6.12.1 Detailed Description

Structure for sensor metadata, which is always included in any packet.

Definition at line 53 of file packets.h.

6.12.2 Member Data Documentation

6.12.2.1 sensor_id

```
uint8_t sensor_metadata::sensor_id
```

ID of the sensor being addressed.

Definition at line 57 of file packets.h.

6.12.2.2 sensor_type

```
SensorType sensor_metadata::sensor_type
```

Type of the sensor being addressed as `SensorType` (one byte)

Definition at line 55 of file `packets.h`.

The documentation for this struct was generated from the following file:

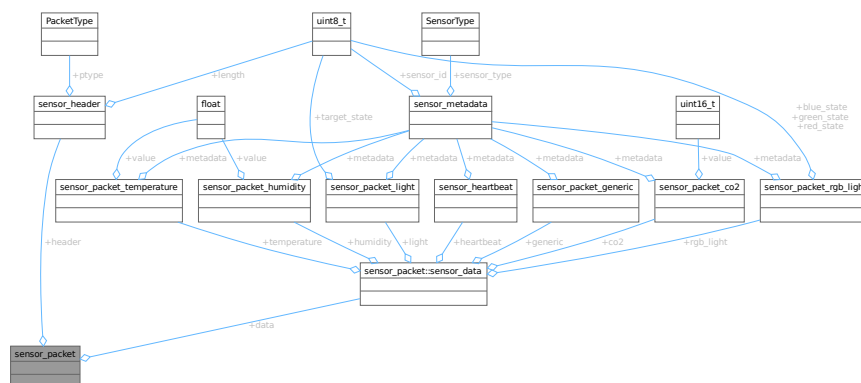
- `include/packets.h`

6.13 sensor_packet Struct Reference

Union structure for the entire sensor packet.

```
#include <packets.h>
```

Collaboration diagram for sensor_packet:



Classes

- union [sensor_data](#)

Public Attributes

- struct [sensor_header](#) header
Header of the packet containing length and type information.
- union [sensor_packet::sensor_data](#) data

6.13.1 Detailed Description

Union structure for the entire sensor packet.

This structure is used to encapsulate the different types of sensor packets that can be sent and has the shape of a valid packet.

It contains a [sensor_header](#) followed by a union of different sensor data types. The union allows for different types of sensor data to be stored in the same memory location, depending on the packet type.

Example usage:

```
sensor_packet packet;
packet.header.length = sizeof(sensor_packet_generic);
packet.header.ptype = PacketType::DATA;
packet.data.generic.metadata.sensor_type = SensorType::BUTTON;
packet.data.generic.metadata.sensor_id = 1;

// Accessing the packet data
if (packet.header.ptype == PacketType::DATA) {
    if (packet.data.generic.metadata.sensor_type == SensorType::BUTTON) {
        uint8_t sensor_id = packet.data.generic.metadata.sensor_id;
        // Process button press event for sensor_id
    }
}
```

To use this structure to request data from the dashboard, you can set the ptype to DASHBOARD_GET to indicate that you want to request data from the backend (wemos bridge). Then, you use a [sensor_packet_generic](#) to specify the type of sensor you want to request data for and the ID of that sensor.

Example: We want to request temperature data from the backend (wemos bridge) for sensor ID 1.

```
sensor_packet packet;
packet.header.length = sizeof(sensor_packet_generic);
packet.header.ptype = PacketType::DASHBOARD_GET;
packet.data.generic.metadata.sensor_type = SensorType::TEMPERATURE;
packet.data.generic.metadata.sensor_id = 1;
```

The backend (wemos bridge) will then respond with a packet of type DASHBOARD_RESPONSE containing the requested data. Following the correct type packet for this example would be a [sensor_packet_temperature](#).

Example: We want to change the color of an RGB light with ID 1 to red (255, 0, 0).

```
sensor_packet packet;
packet.header.length = sizeof(sensor_packet_rgb_light);
packet.header.ptype = PacketType::DASHBOARD_POST;
packet.data.rgb_light.metadata.sensor_type = SensorType::RGB_LIGHT;
packet.data.rgb_light.metadata.sensor_id = 1;
packet.data.rgb_light.red_state = 255;
packet.data.rgb_light.green_state = 0;
packet.data.rgb_light.blue_state = 0;
```

Note

The data field is a union that can hold different types of sensor data.

Definition at line 235 of file [packets.h](#).

6.13.2 Member Data Documentation

6.13.2.1 data

```
union sensor\_packet::sensor\_data sensor_packet::data
```

6.13.2.2 header

```
struct sensor\_header sensor_packet::header
```

Header of the packet containing length and type information.

Definition at line 237 of file [packets.h](#).

The documentation for this struct was generated from the following file:

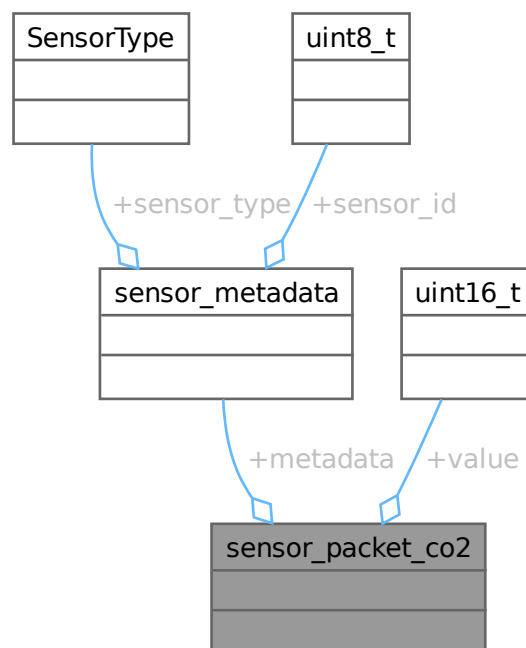
- [include/packets.h](#)

6.14 sensor_packet_co2 Struct Reference

Structure for CO2 sensor packets.

```
#include <packets.h>
```

Collaboration diagram for `sensor_packet_co2`:



Public Attributes

- struct [sensor_metadata](#) [metadata](#)
- [uint16_t](#) [value](#)

Value of the sensor reading the CO2 level represented in ppm.

6.14.1 Detailed Description

Structure for CO2 sensor packets.

This structure contains the type, ID, and value of the CO2 sensor reading.

Note

The CO2 value is represented in parts per million (ppm).

Definition at line [108](#) of file [packets.h](#).

6.14.2 Member Data Documentation

6.14.2.1 metadata

```
struct sensor\_metadata sensor_packet_co2::metadata
```

Definition at line [109](#) of file [packets.h](#).

6.14.2.2 value

```
uint16\_t sensor_packet_co2::value
```

Value of the sensor reading the CO2 level represented in ppm.

Definition at line [111](#) of file [packets.h](#).

The documentation for this struct was generated from the following file:

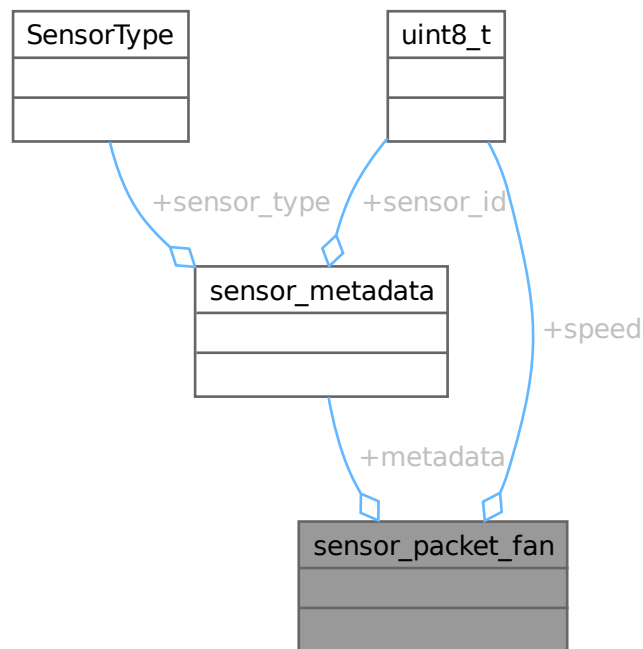
- [include/packets.h](#)

6.15 sensor_packet_fan Struct Reference

Structure for packets intended to control the fan speed.

```
#include <packets.h>
```

Collaboration diagram for sensor_packet_fan:



Public Attributes

- struct [sensor_metadata metadata](#)
- uint8_t [speed](#)

Target speed of the fan (0 -> 255 = 0 -> 100%)

6.15.1 Detailed Description

Structure for packets intended to control the fan speed.

Send the fan speed (0 -> 255) using this packet

Definition at line 164 of file [packets.h](#).

6.15.2 Member Data Documentation

6.15.2.1 metadata

```
struct sensor\_metadata sensor_packet_fan::metadata
```

Definition at line 165 of file [packets.h](#).

6.15.2.2 speed

```
uint8_t sensor_packet_fan::speed
```

Target speed of the fan (0 -> 255 = 0 -> 100%)

Definition at line 167 of file [packets.h](#).

The documentation for this struct was generated from the following file:

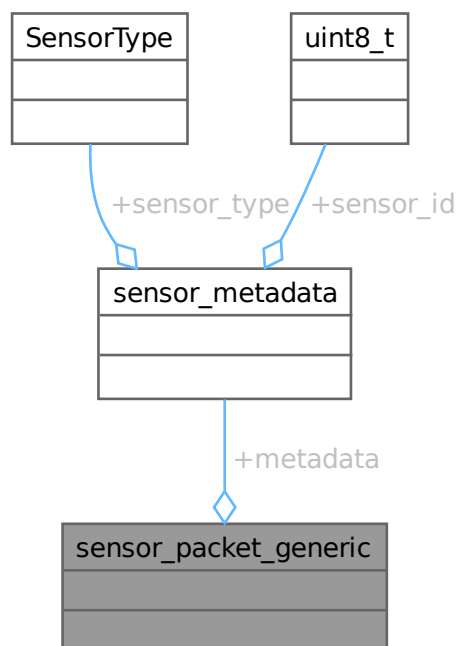
- [include/packets.h](#)

6.16 [sensor_packet_generic](#) Struct Reference

Structure for generic sensor packets.

```
#include <packets.h>
```

Collaboration diagram for [sensor_packet_generic](#):



Public Attributes

- struct [sensor_metadata metadata](#)

6.16.1 Detailed Description

Structure for generic sensor packets.

This structure contains the type and ID of the sensor being addressed. This structure is used for generic sensor packets that do not require additional data. For example, it can be used for a simple button press event.

Definition at line 82 of file [packets.h](#).

6.16.2 Member Data Documentation**6.16.2.1 metadata**

```
struct sensor\_metadata sensor_packet_generic::metadata
```

Definition at line 83 of file [packets.h](#).

The documentation for this struct was generated from the following file:

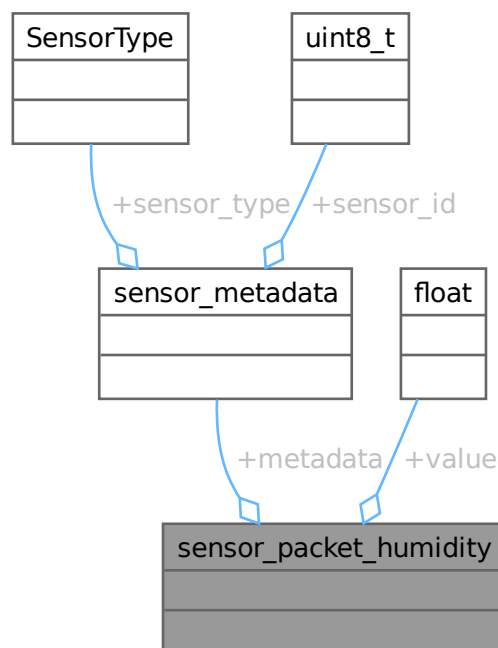
- include/[packets.h](#)

6.17 sensor_packet_humidity Struct Reference

Structure for humidity sensor packets.

```
#include <packets.h>
```

Collaboration diagram for sensor_packet_humidity:



Public Attributes

- struct [sensor_metadata](#) [metadata](#)
- float [value](#)

Value of the sensor reading the humidity level represented in percentage.

6.17.1 Detailed Description

Structure for humidity sensor packets.

This structure contains the type, ID, and value of the humidity sensor reading.

Note

The humidity value is represented in percentage.

Definition at line [121](#) of file [packets.h](#).

6.17.2 Member Data Documentation

6.17.2.1 metadata

```
struct sensor\_metadata sensor_packet_humidity::metadata
```

Definition at line [122](#) of file [packets.h](#).

6.17.2.2 value

```
float sensor_packet_humidity::value
```

Value of the sensor reading the humidity level represented in percentage.

Definition at line [124](#) of file [packets.h](#).

The documentation for this struct was generated from the following file:

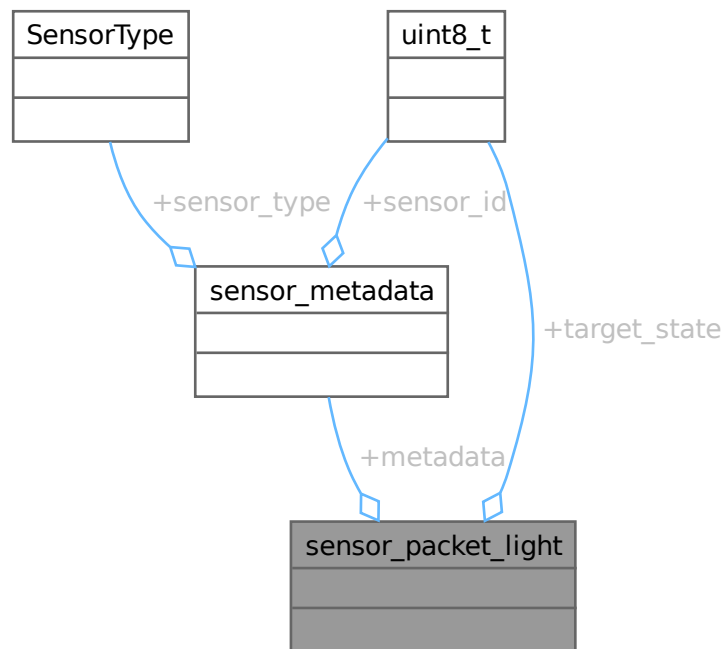
- [include/packets.h](#)

6.18 sensor_packet_light Struct Reference

Structure for light sensor packets.

```
#include <packets.h>
```

Collaboration diagram for sensor_packet_light:



Public Attributes

- struct [sensor_metadata metadata](#)
- `uint8_t` [target_state](#)

Target state of the light (on 1/off 0) represented as a boolean value.

6.18.1 Detailed Description

Structure for light sensor packets.

This structure contains the type, ID, and target state of the light/led. This structure is used for light control packets sent to the light/led.

Definition at line 134 of file [packets.h](#).

6.18.2 Member Data Documentation

6.18.2.1 metadata

struct `sensor_metadata` `sensor_packet_light::metadata`

Definition at line 135 of file [packets.h](#).

6.18.2.2 target_state

uint8_t `sensor_packet_light::target_state`

Target state of the light (on 1/off 0) represented as a boolean value.

Definition at line 137 of file [packets.h](#).

The documentation for this struct was generated from the following file:

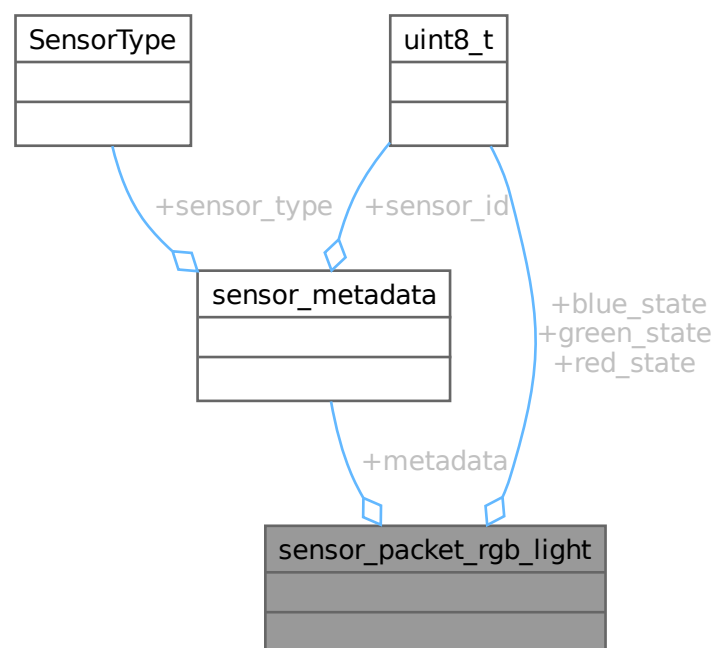
- [include/packets.h](#)

6.19 `sensor_packet_rgb_light` Struct Reference

Structure for RGB light sensor packets.

```
#include <packets.h>
```

Collaboration diagram for `sensor_packet_rgb_light`:



Public Attributes

- struct [sensor_metadata](#) [metadata](#)
- uint8_t [red_state](#)
Target state of the red color (0-255) represented as an 8-bit integer.
- uint8_t [green_state](#)
Target state of the green color (0-255) represented as an 8-bit integer.
- uint8_t [blue_state](#)
Target state of the blue color (0-255) represented as an 8-bit integer.

6.19.1 Detailed Description

Structure for RGB light sensor packets.

This structure contains the type, ID, and target color of the RGB light. This structure is used for RGB light control packets sent to the RGB light.

Note

The RGB values are represented as 8-bit integers (0-255).

Definition at line 148 of file [packets.h](#).

6.19.2 Member Data Documentation

6.19.2.1 blue_state

```
uint8_t sensor_packet_rgb_light::blue_state
```

Target state of the blue color (0-255) represented as an 8-bit integer.

Definition at line 155 of file [packets.h](#).

6.19.2.2 green_state

```
uint8_t sensor_packet_rgb_light::green_state
```

Target state of the green color (0-255) represented as an 8-bit integer.

Definition at line 153 of file [packets.h](#).

6.19.2.3 metadata

```
struct sensor\_metadata sensor_packet_rgb_light::metadata
```

Definition at line 149 of file [packets.h](#).

6.19.2.4 red_state

```
uint8_t sensor_packet_rgb_light::red_state
```

Target state of the red color (0-255) represented as an 8-bit integer.

Definition at line 151 of file [packets.h](#).

The documentation for this struct was generated from the following file:

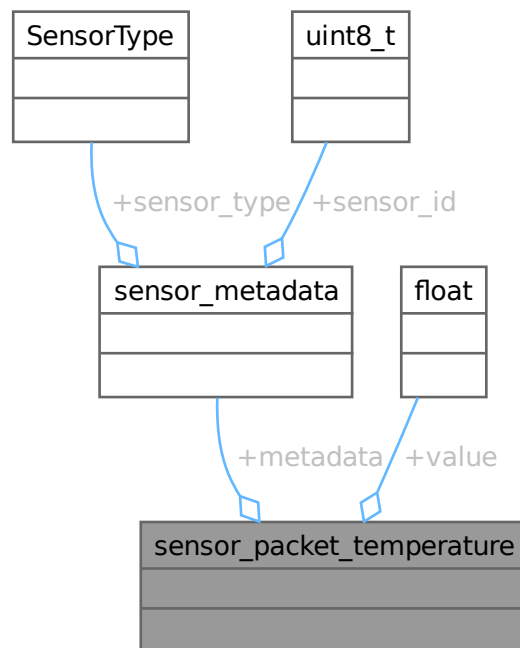
- [include/packets.h](#)

6.20 sensor_packet_temperature Struct Reference

Structure for temperature sensor packets.

```
#include <packets.h>
```

Collaboration diagram for sensor_packet_temperature:



Public Attributes

- struct [sensor_metadata metadata](#)
- float [value](#)

Value of the sensor reading the temperature represented in Celcius.

6.20.1 Detailed Description

Structure for temperature sensor packets.

This structure contains the type, ID, and value of the temperature sensor reading.

Note

The temperature value is represented in Celsius.

Definition at line 95 of file [packets.h](#).

6.20.2 Member Data Documentation

6.20.2.1 metadata

```
struct sensor\_metadata sensor_packet_temperature::metadata
```

Definition at line 96 of file [packets.h](#).

6.20.2.2 value

```
float sensor_packet_temperature::value
```

Value of the sensor reading the temperature represented in Celcius.

Definition at line 98 of file [packets.h](#).

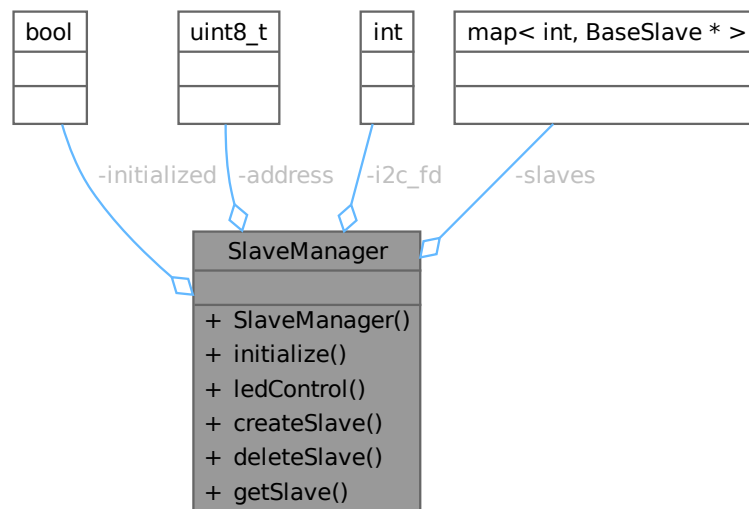
The documentation for this struct was generated from the following file:

- [include/packets.h](#)

6.21 SlaveManager Class Reference

```
#include <SlaveManager.h>
```

Collaboration diagram for SlaveManager:



Public Member Functions

- [SlaveManager](#) ()
- void [initialize](#) ()
Setup the underlying I2C bus.
- void [ledControl](#) (uint8_t led_number, uint8_t led_state)
- void [createSlave](#) ([SensorType](#) type, int i2c_address)
- void [deleteSlave](#) (uint8_t id)
Unmaps the slave from the internal mapping.
- [BaseSlave](#) * [getSlave](#) (int id)
Get a slavedevice with a given ID.

Private Attributes

- bool [initialized](#) = false
- uint8_t [address](#) = 0x01
- int [i2c_fd](#) = -1
- std::map< int, [BaseSlave](#) * > [slaves](#)

6.21.1 Detailed Description

Definition at line 9 of file [SlaveManager.h](#).

6.21.2 Constructor & Destructor Documentation

6.21.2.1 SlaveManager()

```
SlaveManager::SlaveManager ( )
```

Definition at line 15 of file [SlaveManager.cpp](#).

6.21.3 Member Function Documentation

6.21.3.1 createSlave()

```
void SlaveManager::createSlave (
    SensorType type,
    int i2c_address )
```

Definition at line 30 of file [SlaveManager.cpp](#).

6.21.3.2 deleteSlave()

```
void SlaveManager::deleteSlave (
    uint8_t id )
```

Unmaps the slave from the internal mapping.

Parameters

<i>id</i>	The ID of the slave device to unmap
-----------	-------------------------------------

Definition at line 47 of file [SlaveManager.cpp](#).

6.21.3.3 getSlave()

```
BaseSlave * SlaveManager::getSlave (
    int id )
```

Get a slavedevice with a given ID.

Parameters

<i>id</i>	The ID of the slave device to retrieve
-----------	--

Definition at line 56 of file [SlaveManager.cpp](#).

6.21.3.4 initialize()

```
void SlaveManager::initialize ( )
```

Setup the underlying I2C bus.

Exceptions

<i>std__runtime_error</i>	if the I2C setup fails
---------------------------	------------------------

Definition at line 17 of file [SlaveManager.cpp](#).

6.21.3.5 ledControl()

```
void SlaveManager::ledControl (
    uint8_t led_number,
    uint8_t led_state )
```

Definition at line 28 of file [SlaveManager.cpp](#).

6.21.4 Member Data Documentation

6.21.4.1 address

```
uint8_t SlaveManager::address = 0x01 [private]
```

Definition at line 46 of file [SlaveManager.h](#).

6.21.4.2 i2c_fd

```
int SlaveManager::i2c_fd = -1 [private]
```

Definition at line 47 of file [SlaveManager.h](#).

6.21.4.3 initialized

```
bool SlaveManager::initialized = false [private]
```

Definition at line 44 of file [SlaveManager.h](#).

6.21.4.4 slaves

```
std::map<int, BaseSlave*> SlaveManager::slaves [private]
```

Definition at line 49 of file [SlaveManager.h](#).

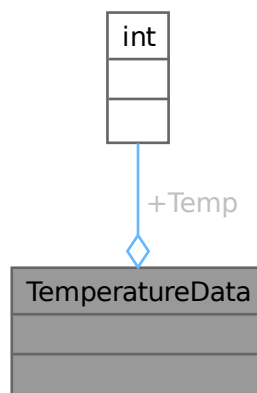
The documentation for this class was generated from the following files:

- [include/SlaveManager.h](#)
- [src/SlaveManager.cpp](#)

6.22 TemperatureData Struct Reference

```
#include <TemperatureSlave.h>
```

Collaboration diagram for TemperatureData:



Public Attributes

- int [Temp](#)

6.22.1 Detailed Description

Definition at line 6 of file [TemperatureSlave.h](#).

6.22.2 Member Data Documentation

6.22.2.1 Temp

```
int TemperatureData::Temp
```

Definition at line 7 of file [TemperatureSlave.h](#).

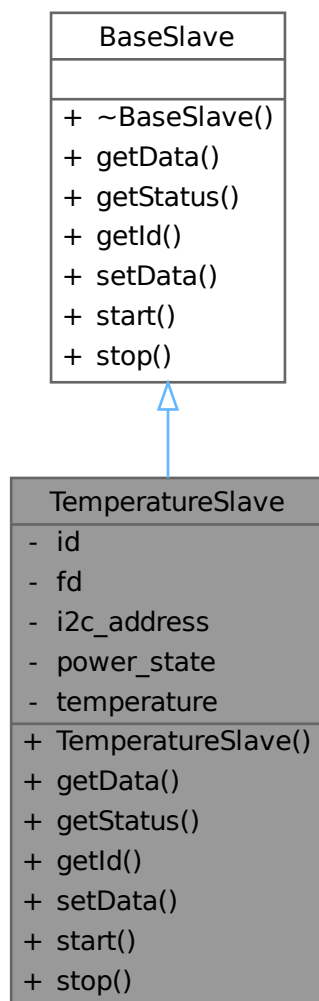
The documentation for this struct was generated from the following file:

- include/[TemperatureSlave.h](#)

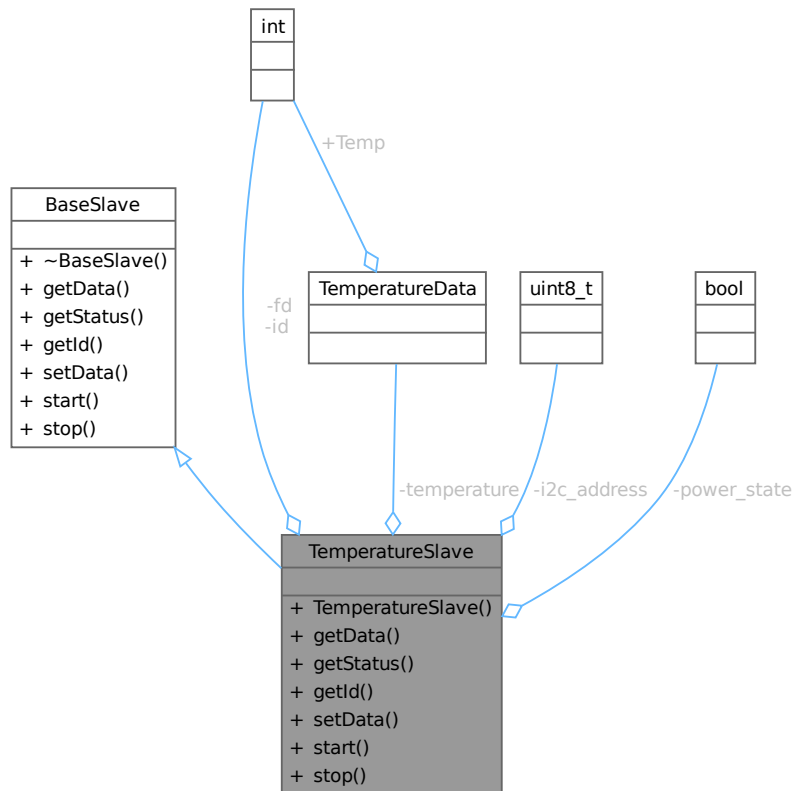
6.23 TemperatureSlave Class Reference

```
#include <TemperatureSlave.h>
```

Inheritance diagram for TemperatureSlave:



Collaboration diagram for TemperatureSlave:



Public Member Functions

- `TemperatureSlave` (`uint8_t id`, `uint8_t i2c_address`)
- `void * getData` ()
- `bool getStatus` ()
- `int getId` ()
- `void setData` (`void *data`)
- `void start` (`int i2c_fd`)
- `void stop` ()

Public Member Functions inherited from `BaseSlave`

- virtual `~BaseSlave` ()=default

Private Attributes

- `int id`
- `int fd`
- `uint8_t i2c_address`
- `bool power_state`
- `TemperatureData temperature`

6.23.1 Detailed Description

Definition at line 10 of file [TemperatureSlave.h](#).

6.23.2 Constructor & Destructor Documentation

6.23.2.1 TemperatureSlave()

```
TemperatureSlave::TemperatureSlave (
    uint8_t id,
    uint8_t i2c_address )
```

Definition at line 3 of file [TemperatureSlave.cpp](#).

6.23.3 Member Function Documentation

6.23.3.1 getData()

```
void * TemperatureSlave::getData ( ) [virtual]
```

Implements [BaseSlave](#).

Definition at line 6 of file [TemperatureSlave.cpp](#).

6.23.3.2 getId()

```
int TemperatureSlave::getId ( ) [virtual]
```

Implements [BaseSlave](#).

Definition at line 23 of file [TemperatureSlave.cpp](#).

6.23.3.3 getStatus()

```
bool TemperatureSlave::getStatus ( ) [virtual]
```

Implements [BaseSlave](#).

Definition at line 14 of file [TemperatureSlave.cpp](#).

6.23.3.4 setData()

```
void TemperatureSlave::setData (
    void * data ) [virtual]
```

Implements [BaseSlave](#).

Definition at line 25 of file [TemperatureSlave.cpp](#).

6.23.3.5 start()

```
void TemperatureSlave::start (
    int i2c_fd ) [virtual]
```

Implements [BaseSlave](#).

Definition at line 32 of file [TemperatureSlave.cpp](#).

6.23.3.6 stop()

```
void TemperatureSlave::stop ( ) [virtual]
```

Implements [BaseSlave](#).

Definition at line 34 of file [TemperatureSlave.cpp](#).

6.23.4 Member Data Documentation

6.23.4.1 fd

```
int TemperatureSlave::fd [private]
```

Definition at line 22 of file [TemperatureSlave.h](#).

6.23.4.2 i2c_address

```
uint8_t TemperatureSlave::i2c_address [private]
```

Definition at line 23 of file [TemperatureSlave.h](#).

6.23.4.3 id

```
int TemperatureSlave::id [private]
```

Definition at line 21 of file [TemperatureSlave.h](#).

6.23.4.4 power_state

```
bool TemperatureSlave::power_state [private]
```

Definition at line 25 of file [TemperatureSlave.h](#).

6.23.4.5 temperature

```
TemperatureData TemperatureSlave::temperature [private]
```

Definition at line 26 of file [TemperatureSlave.h](#).

The documentation for this class was generated from the following files:

- include/[TemperatureSlave.h](#)
- src/[TemperatureSlave.cpp](#)

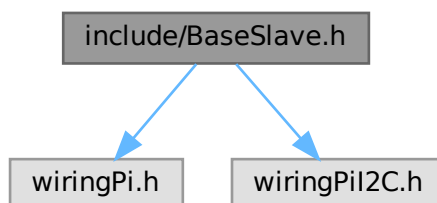
Chapter 7

File Documentation

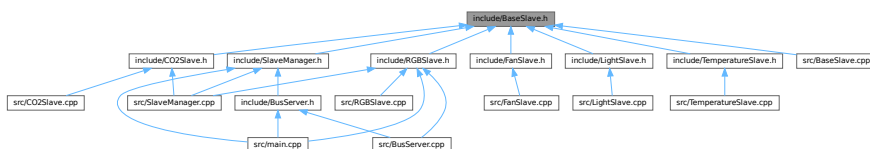
7.1 include/BaseSlave.h File Reference

```
#include <wiringPi.h>
#include <wiringPiI2C.h>
```

Include dependency graph for BaseSlave.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [BaseSlave](#)

7.2 BaseSlave.h

[Go to the documentation of this file.](#)

```

00001 #pragma once
00002
00003 #include <wiringPi.h>
00004 #include <wiringPiI2C.h>
00005
00006 class BaseSlave {
00007     public:
00008         virtual ~BaseSlave() = default;
00009         virtual const void* getData() = 0;
00010         virtual bool getStatus() = 0;
00011         virtual int getId() = 0;
00012         virtual void setData(void* data) = 0;
00013         virtual void start(int i2c_fd) = 0;
00014         virtual void stop() = 0;
00015 };

```

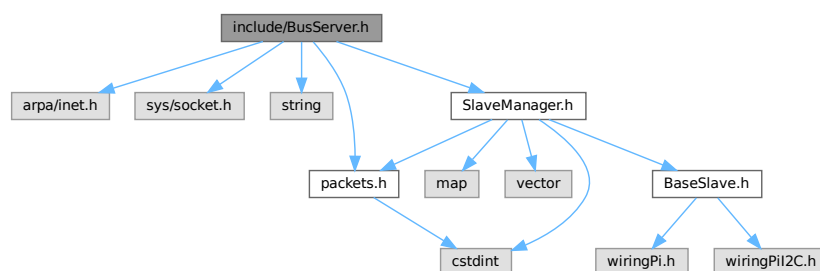
7.3 include/BusServer.h File Reference

```

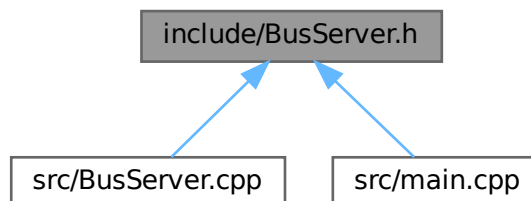
#include <arpa/inet.h>
#include <sys/socket.h>
#include <string>
#include "SlaveManager.h"
#include "packets.h"

```

Include dependency graph for BusServer.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [BusServer](#)

Macros

- `#define BUFFER_SIZE 1024`

7.3.1 Macro Definition Documentation

7.3.1.1 BUFFER_SIZE

```
#define BUFFER_SIZE 1024
```

Definition at line 10 of file [BusServer.h](#).

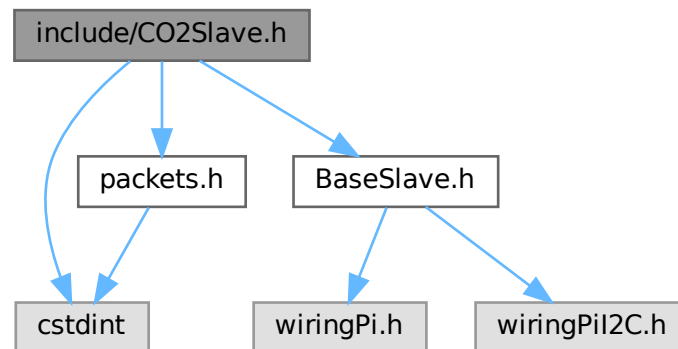
7.4 BusServer.h

[Go to the documentation of this file.](#)

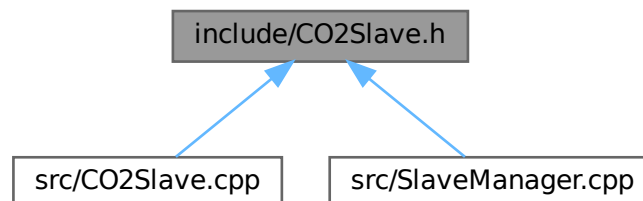
```
00001 #pragma once
00002 #include <arpa/inet.h>
00003 #include <sys/socket.h>
00004
00005 #include <string>
00006
00007 #include "SlaveManager.h"
00008 #include "packets.h"
00009
00010 #define BUFFER_SIZE 1024
00011
00012 class BusServer {
00013     public:
00014         BusServer() : listening_fd(-1) {}
00015
00025         void setup(std::string ip, int port);
00026
00031         void listen();
00032
00041         void send(struct sensor_packet* pkt, int fd);
00042
00048         void start();
00049
00053         SlaveManager& getSlaveManager();
00054
00055     private:
00056         int listening_fd;
00057
00058         struct sockaddr_in listening_address;
00059         bool wemos_bridge_connected = false;
00060         char buffer[BUFFER_SIZE];
00061
00062         SlaveManager slave_manager;
00063 };
```

7.5 include/CO2Slave.h File Reference

```
#include <cstdint>
#include "BaseSlave.h"
#include "packets.h"
Include dependency graph for CO2Slave.h:
```



This graph shows which files directly or indirectly include this file:



Classes

- class [CO2Slave](#)

7.6 CO2Slave.h

[Go to the documentation of this file.](#)

```
00001 #pragma once
00002 #include <cstdint>
00003
00004 #include "BaseSlave.h"
```

```

00005 #include "packets.h"
00006
00007 class CO2Slave : public BaseSlave {
00008     public:
00009         CO2Slave(uint8_t id, uint8_t i2c_address);
00010
00011         const void* getData() override;
00012         bool getStatus() override;
00013         int getId() override;
00014         void setData(void* data) override;
00015         void start(int i2c_fd) override;
00016         void stop() override;
00017
00018     private:
00019         int id;
00020         int fd;
00021         uint8_t i2c_address;
00022         uint16_t command;
00023
00024         bool power_state = true;
00025         struct sensor_packet state_packet;
00026 };

```

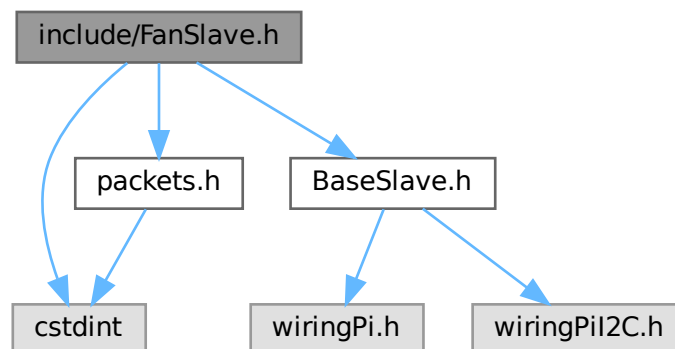
7.7 include/FanSlave.h File Reference

```

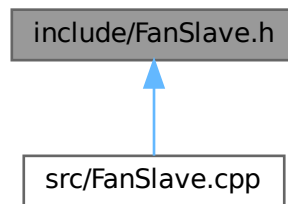
#include <stdint>
#include "BaseSlave.h"
#include "packets.h"

```

Include dependency graph for FanSlave.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [FanSlave](#)

Typedefs

- typedef uint8_t [FanData](#)

7.7.1 Typedef Documentation

7.7.1.1 FanData

```
typedef uint8_t FanData
```

Definition at line 8 of file [FanSlave.h](#).

7.8 FanSlave.h

[Go to the documentation of this file.](#)

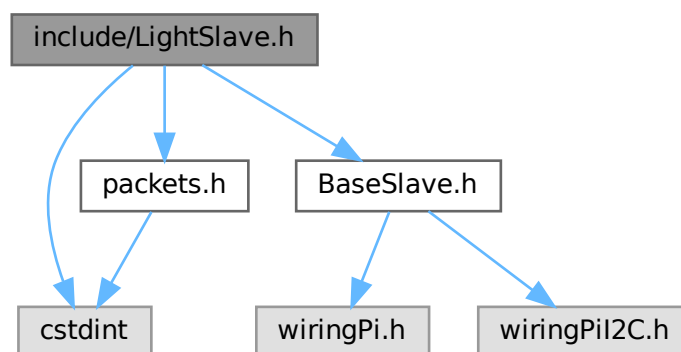
```

00001 #pragma once
00002
00003 #include <stdint>
00004
00005 #include "BaseSlave.h"
00006 #include "packets.h"
00007
00008 typedef uint8_t FanData;
00009
00010 class FanSlave : public BaseSlave {
00011     public:
00012         FanSlave(uint8_t id, uint8_t i2c_address);
00013         const void* getData();
00014         bool getStatus();
00015         int getId();
00016         void setData(void* data);
00017         void start(int i2c_fd);
00018         void stop();
00019
00020     private:
00021         int id;
00022         int fd;
00023         uint8_t i2c_address;
00024
00025         FanData current_speed;
00026
00027         struct sensor_packet state_packet;
00028 };
  
```

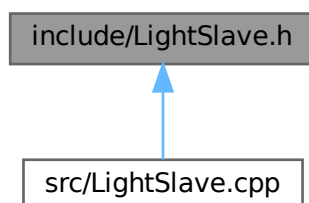

7.9 include/LightSlave.h File Reference

```
#include <stdint>
#include "BaseSlave.h"
#include "packets.h"
```

Include dependency graph for LightSlave.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [LightSlave](#)

Typedefs

- typedef uint8_t [LightData](#)

7.9.1 Typedef Documentation

7.9.1.1 LightData

```
typedef uint8_t LightData
```

Definition at line 7 of file [LightSlave.h](#).

7.10 LightSlave.h

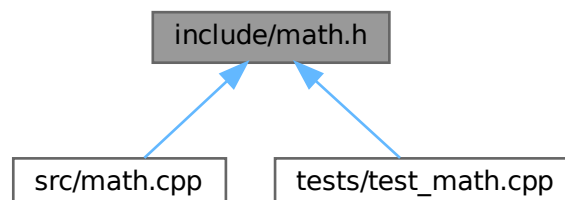
[Go to the documentation of this file.](#)

```
00001 #pragma once
00002 #include <stdint>
00003
00004 #include "BaseSlave.h"
00005 #include "packets.h"
00006
00007 typedef uint8_t LightData;
00008
00009 class LightSlave : public BaseSlave {
00010     public:
00011         LightSlave(uint8_t id, uint8_t i2c_address);
00012         const void* getData();
00013         bool getStatus();
00014         int getId();
00015         void setData(void* data);
00016         void start(int i2c_fd);
00017         void stop();
00018
00019     private:
00020         int id;
00021         int fd;
00022         uint8_t i2c_address;
00023
00024         LightData color_state;
00025
00026         struct sensor_packet state_packet;
00027 };
```

7.11 include/math.h File Reference

Header file for [math.cpp](#).

This graph shows which files directly or indirectly include this file:



Functions

- `int add (int a, int b)`
Adds two integers.
- `int subtract (int a, int b)`
Subtracts two integers.

7.11.1 Detailed Description

Header file for [math.cpp](#).

This file contains declarations for basic math operations.

Author

Daan Breur

Definition in file [math.h](#).

7.11.2 Function Documentation

7.11.2.1 add()

```
int add (  
    int a,  
    int b )
```

Adds two integers.

Parameters

<i>a</i>	First integer.
<i>b</i>	Second integer.

Returns

The sum of a and b.

This function takes two integers as input and returns their sum.

Definition at line 16 of file [math.cpp](#).

7.11.2.2 subtract()

```
int subtract (  
    int a,  
    int b )
```

Subtracts two integers.

Parameters

<i>a</i>	First integer.
<i>b</i>	Second integer.

Returns

The difference of a and b.

This function takes two integers as input and returns the result of subtracting b from a.

Definition at line 26 of file [math.cpp](#).

7.12 math.h

[Go to the documentation of this file.](#)

```

00001
00008 #ifndef MATH_H
00009 #define MATH_H
00010
00011 int add(int a, int b);
00012 int subtract(int a, int b);
00013
00014 #endif

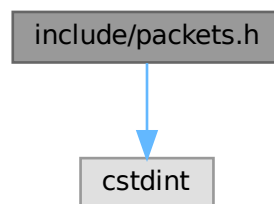
```

7.13 include/packets.h File Reference

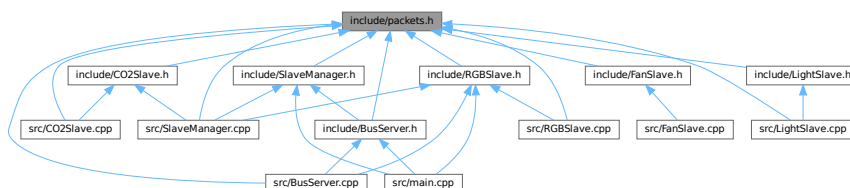
Header file for [packets.h](#).

```
#include <cstdint>
```

Include dependency graph for packets.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct [sensor_header](#)
Header structure for sensor packets.
- struct [sensor_metadata](#)
Structure for sensor metadata, which is always included in any packet.
- struct [sensor_heartbeat](#)
Structure for heartbeat packets.
- struct [sensor_packet_generic](#)
Structure for generic sensor packets.
- struct [sensor_packet_temperature](#)
Structure for temperature sensor packets.
- struct [sensor_packet_co2](#)
Structure for CO2 sensor packets.
- struct [sensor_packet_humidity](#)
Structure for humidity sensor packets.
- struct [sensor_packet_light](#)
Structure for light sensor packets.
- struct [sensor_packet_rgb_light](#)
Structure for RGB light sensor packets.
- struct [sensor_packet_fan](#)
Structure for packets intended to control the fan speed.
- struct [sensor_packet](#)
Union structure for the entire sensor packet.
- union [sensor_packet::sensor_data](#)

- union [sensor_data](#)

Enumerations

- enum class [SensorType](#) : uint8_t {
 [NOOP](#) = 0 , [BUTTON](#) = 1 , [TEMPERATURE](#) = 2 , [CO2](#) = 3 ,
 [HUMIDITY](#) = 4 , [PRESSURE](#) = 5 , [LIGHT](#) = 6 , [MOTION](#) = 7 ,
 [RGB_LIGHT](#) = 8 , [FAN](#) = 10 }
- enum class [PacketType](#) : uint8_t {
 [DATA](#) = 0 , [HEARTBEAT](#) = 1 , [DASHBOARD_POST](#) = 2 , [DASHBOARD_GET](#) = 3 ,
 [DASHBOARD_RESPONSE](#) = 4 }

Functions

- struct [sensor_header __attribute__\(\(packed\)\)](#)

Variables

- `uint8_t length`
Length of the packet excluding the header.
- `PacketType ptype`
Type of the packet as `PacketType` (`DATA`, `HEARTBEAT`, etc.).
- `SensorType sensor_type`
Type of the sensor being addressed as `SensorType` (one byte)
- `uint8_t sensor_id`
ID of the sensor being addressed.
- `struct sensor_metadata metadata`
- `float value`
Value of the sensor reading the temperature represented in Celcius.
- `uint8_t target_state`
Target state of the light (on 1/off 0) represented as a boolean value.
- `uint8_t red_state`
Target state of the red color (0-255) represented as an 8-bit integer.
- `uint8_t green_state`
Target state of the green color (0-255) represented as an 8-bit integer.
- `uint8_t blue_state`
Target state of the blue color (0-255) represented as an 8-bit integer.
- `uint8_t speed`
Target speed of the fan (0 -> 255 = 0 -> 100%)
- `struct sensor_header header`
Header of the packet containing length and type information.
- `union sensor_data data`

7.13.1 Detailed Description

Header file for `packets.h`.

This files origin is from the Wemos project

Warning

THIS FILE MUST BE KEPT IN SYNC IN OTHER PROJECTS

Author

Daan Breur
Erynn Scholtes

Definition in file `packets.h`.

7.13.2 Enumeration Type Documentation

7.13.2.1 PacketType

```
enum class PacketType : uint8_t [strong]
```

Enumerator

DATA	
HEARTBEAT	
DASHBOARD_POST	
DASHBOARD_GET	
DASHBOARD_RESPONSE	

Definition at line 28 of file [packets.h](#).

7.13.2.2 SensorType

```
enum class SensorType : uint8_t [strong]
```

Enumerator

NOOP	
BUTTON	
TEMPERATURE	
CO2	
HUMIDITY	
PRESSURE	
LIGHT	
MOTION	
RGB_LIGHT	
FAN	

Definition at line 15 of file [packets.h](#).

7.13.3 Function Documentation

7.13.3.1 __attribute__()

```
struct sensor\_header __attribute__ (  
    (packed) )
```

7.13.4 Variable Documentation

7.13.4.1 blue_state

```
uint8_t blue_state
```

Target state of the blue color (0-255) represented as an 8-bit integer.

Definition at line 6 of file [packets.h](#).

7.13.4.2 data

```
union sensor\_data data
```

7.13.4.3 green_state

```
uint8_t green_state
```

Target state of the green color (0-255) represented as an 8-bit integer.

Definition at line 4 of file [packets.h](#).

7.13.4.4 header

```
struct sensor\_header header
```

Header of the packet containing length and type information.

Definition at line 1 of file [packets.h](#).

7.13.4.5 length

```
uint8_t length
```

Length of the packet excluding the header.

Definition at line 1 of file [packets.h](#).

7.13.4.6 metadata

```
struct sensor\_metadata metadata
```

Definition at line 0 of file [packets.h](#).

7.13.4.7 ptype

```
PacketType ptype
```

Type of the packet as PacketType (DATA, HEARTBEAT, etc.).

Definition at line 3 of file [packets.h](#).

7.13.4.8 red_state

```
uint8_t red_state
```

Target state of the red color (0-255) represented as an 8-bit integer.

Definition at line 2 of file [packets.h](#).

7.13.4.9 sensor_id

```
uint8_t sensor_id
```

ID of the sensor being addressed.

Definition at line 3 of file [packets.h](#).

7.13.4.10 sensor_type

```
SensorType sensor_type
```

Type of the sensor being addressed as SensorType (one byte)

Definition at line 1 of file [packets.h](#).

7.13.4.11 speed

```
uint8_t speed
```

Target speed of the fan (0 -> 255 = 0 -> 100%)

Definition at line 2 of file [packets.h](#).

7.13.4.12 target_state

```
uint8_t target_state
```

Target state of the light (on 1/off 0) represented as a boolean value.

Definition at line 2 of file [packets.h](#).

7.13.4.13 value

```
float value
```

Value of the sensor reading the temperature represented in Celcius.

Value of the sensor reading the humidity level represented in percentage.

Value of the sensor reading the CO2 level represented in ppm.

Definition at line 2 of file [packets.h](#).

7.14 packets.h

[Go to the documentation of this file.](#)

```

00001
00010 #ifndef PACKETS_H
00011 #define PACKETS_H
00012
00013 #include <stdint>
00014
00015 enum class SensorType : uint8_t {
00016     NOOP = 0,
00017     BUTTON = 1,
00018     TEMPERATURE = 2,
00019     CO2 = 3,
00020     HUMIDITY = 4,
00021     PRESSURE = 5,
00022     LIGHT = 6,
00023     MOTION = 7,
00024     RGB_LIGHT = 8,
00025     FAN = 10,
00026 };
00027
00028 enum class PacketType : uint8_t {
00029     DATA = 0,
00030     HEARTBEAT = 1,
00031     DASHBOARD_POST = 2,
00032     DASHBOARD_GET = 3,
00033     DASHBOARD_RESPONSE = 4
00034 };
00035
00041 struct sensor_header {
00043     uint8_t length;
00045     PacketType ptype;
00046 } __attribute__((packed));
00047
00053 struct sensor_metadata {
00055     SensorType sensor_type;
00057     uint8_t sensor_id;
00058 } __attribute__((packed));
00059
00060 // Specific packet structures (ensure alignment/packing matches expected format)
00061
00070 struct sensor_heartbeat {
00071     struct sensor_metadata metadata;
00072 } __attribute__((packed));
00073
00082 struct sensor_packet_generic {
00083     struct sensor_metadata metadata;
00084     // /** @brief Whether the sensor did or did not trigger */
00085     // bool value;
00086 } __attribute__((packed));
00087
00095 struct sensor_packet_temperature {
00096     struct sensor_metadata metadata;
00098     float value;
00099 } __attribute__((packed));
00100
00108 struct sensor_packet_co2 {
00109     struct sensor_metadata metadata;
00111     uint16_t value;
00112 } __attribute__((packed));
00113
00121 struct sensor_packet_humidity {
00122     struct sensor_metadata metadata;
00124     float value;
00125 } __attribute__((packed));
00126
00134 struct sensor_packet_light {
00135     struct sensor_metadata metadata;
00137     uint8_t target_state;
00138 } __attribute__((packed));
00139
00148 struct sensor_packet_rgb_light {
00149     struct sensor_metadata metadata;
00151     uint8_t red_state;
00153     uint8_t green_state;
00155     uint8_t blue_state;
00156 } __attribute__((packed));
00157
00164 struct sensor_packet_fan {
00165     struct sensor_metadata metadata;
00167     uint8_t speed;
00168 } __attribute__((packed));
00169 // --- End Structures ---
00170

```

```

00235 struct sensor_packet {
00237     struct sensor_header header;
00238
00240     union sensor_data {
00241         struct sensor_heartbeat heartbeat;
00242         struct sensor_packet_generic generic;
00243         struct sensor_packet_temperature temperature;
00244         struct sensor_packet_co2 co2;
00245         struct sensor_packet_humidity humidity;
00246         struct sensor_packet_light light;
00247         struct sensor_packet_rgb_light rgb_light;
00248     } data;
00249 } __attribute__((packed));
00250
00251 #endif // PACKETS_H

```

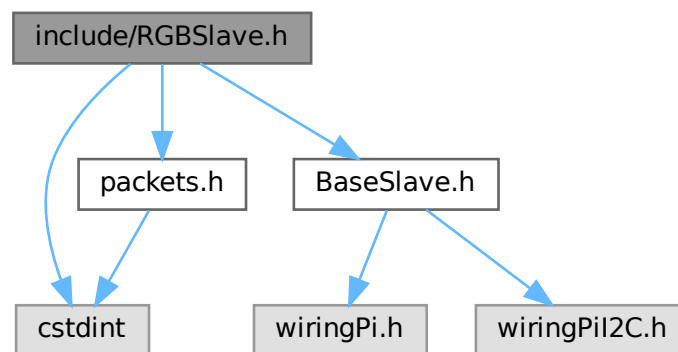
7.15 include/RGBSlave.h File Reference

```
#include <cstdint>
```

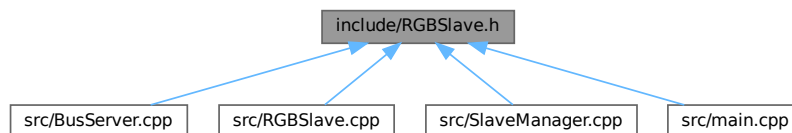
```
#include "BaseSlave.h"
```

```
#include "packets.h"
```

Include dependency graph for RGBSlave.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct [RGBData](#)
- class [RGBSlave](#)

Functions

- struct [RGBData __attribute__](#) ((packed))

Variables

- uint8_t [R](#)
- uint8_t [G](#)
- uint8_t [B](#)
- [RGBSlave __attribute__](#)

7.15.1 Function Documentation

7.15.1.1 [__attribute__](#)()

```
struct RGBData \_\_attribute\_\_ (  
    (packed) )
```

7.15.2 Variable Documentation

7.15.2.1 [__attribute__](#)

```
struct sensor\_packet \_\_attribute\_\_
```

7.15.2.2 [B](#)

```
uint8_t B
```

Definition at line 0 of file [RGBSlave.h](#).

7.15.2.3 [G](#)

```
uint8_t G
```

Definition at line 0 of file [RGBSlave.h](#).

7.15.2.4 [R](#)

```
uint8_t R
```

Definition at line 0 of file [RGBSlave.h](#).

7.16 RGBSlave.h

[Go to the documentation of this file.](#)

```

00001 #pragma once
00002 #include <stdint>
00003
00004 #include "BaseSlave.h"
00005 #include "packets.h"
00006
00007 struct RGBData {
00008     uint8_t R, G, B;
00009 } __attribute__((packed));
00010
00011 class RGBSlave : public BaseSlave {
00012 public:
00013     RGBSlave(uint8_t id, uint8_t i2c_address);
00014     const void* getData();
00015     bool getStatus();
00016     int getId();
00017     void setData(void* data);
00018     void start(int i2c_fd);
00019     void stop();
00020
00021 private:
00022     int id;
00023     int fd;
00024     uint8_t i2c_address;
00025
00026     RGBData color_state;
00027
00028     struct sensor_packet state_packet;
00029 };

```

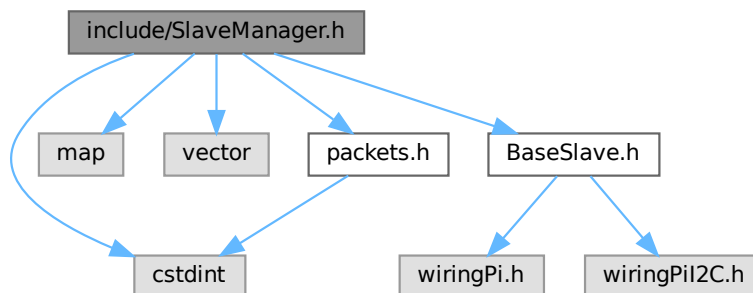
7.17 include/SlaveManager.h File Reference

```

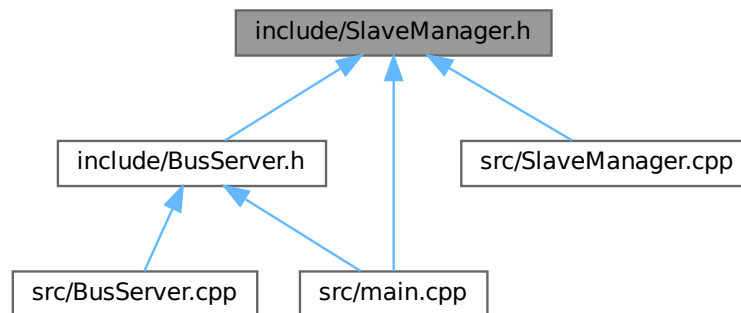
#include <stdint>
#include <map>
#include <vector>
#include "BaseSlave.h"
#include "packets.h"

```

Include dependency graph for SlaveManager.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [SlaveManager](#)

7.18 SlaveManager.h

[Go to the documentation of this file.](#)

```

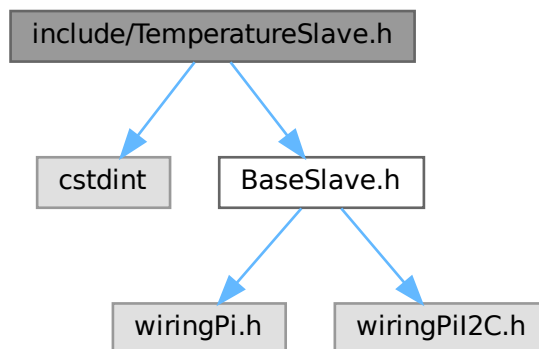
00001 #pragma once
00002 #include <stdint>
00003 #include <map>
00004 #include <vector>
00005
00006 #include "BaseSlave.h"
00007 #include "packets.h"
00008
00009 class SlaveManager {
00010 public:
00011     SlaveManager();
00012
00017     void initialize();
00018
00019     void ledControl(uint8_t led_number, uint8_t led_state);
00020
00021     /*
00022      * @brief Creates a new slave device into the internal mapping
00023      * @details Assigns a pre-known SensorType and ID to an I2C device, and then opens the I2C
00024      * connection to it
00025      * @param type The SensorType of the sensor, as defined in packets.h
00026      * @param id The ID of the slave device
00027      * @param i2c_address The I2C address of the slave device
00028      */
00029     void createSlave(SensorType type, int i2c_address);
00030
00035     void deleteSlave(uint8_t id);
00036
00041     BaseSlave* getSlave(int id);
00042
00043 private:
00044     bool initialized = false;
00045
00046     uint8_t address = 0x01;
00047     int i2c_fd = -1;
00048
00049     std::map<int, BaseSlave*> slaves; // maps ID to BaseSlave ptr
00050 };
  
```

7.19 include/TemperatureSlave.h File Reference

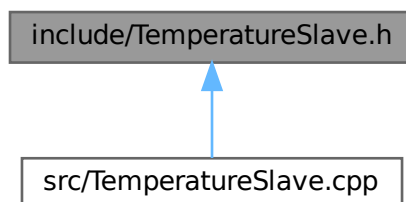
```
#include <stdint>
```

```
#include "BaseSlave.h"
```

Include dependency graph for TemperatureSlave.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct [TemperatureData](#)
- class [TemperatureSlave](#)

Functions

- struct [TemperatureData](#) `__attribute__((packed))`

Variables

- int [Temp](#)
- [TemperatureSlave](#) `__attribute__((packed))`

7.19.1 Function Documentation

7.19.1.1 `__attribute__()`

```
struct TemperatureData __attribute__ (
    (packed) )
```

7.19.2 Variable Documentation

7.19.2.1 `__attribute__`

```
TemperatureSlave __attribute__
```

7.19.2.2 `Temp`

```
int Temp
```

Definition at line 0 of file [TemperatureSlave.h](#).

7.20 TemperatureSlave.h

[Go to the documentation of this file.](#)

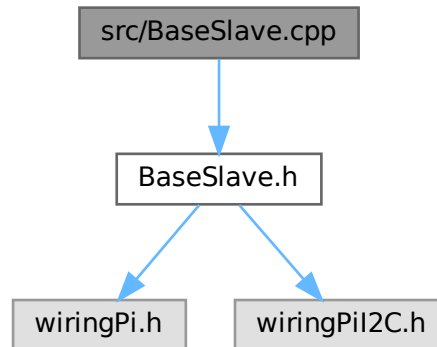
```
00001 #pragma once
00002 #include <stdint>
00003
00004 #include "BaseSlave.h"
00005
00006 struct TemperatureData {
00007     int Temp;
00008 } __attribute__((packed));
00009
00010 class TemperatureSlave : public BaseSlave {
00011     public:
00012         TemperatureSlave(uint8_t id, uint8_t i2c_address);
00013         void* getData();
00014         bool getStatus();
00015         int getId();
00016         void setData(void* data);
00017         void start(int i2c_fd);
00018         void stop();
00019
00020     private:
00021         int id;
00022         int fd;
00023         uint8_t i2c_address;
00024
00025         bool power_state;
00026         TemperatureData temperature;
00027 };
```


7.21 README.md File Reference

7.22 src/BaseSlave.cpp File Reference

```
#include "BaseSlave.h"
```

Include dependency graph for BaseSlave.cpp:



7.23 BaseSlave.cpp

[Go to the documentation of this file.](#)

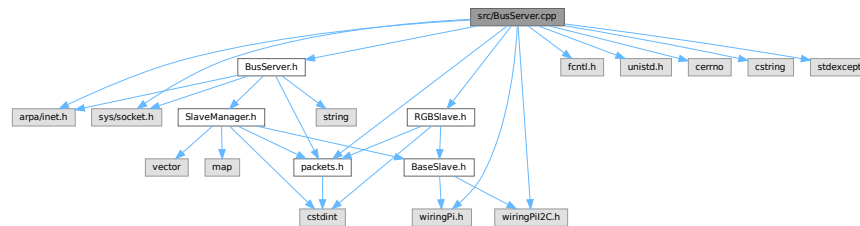
```
00001 #include "BaseSlave.h"
```

7.24 src/BusServer.cpp File Reference

```
#include "BusServer.h"  
#include <arpa/inet.h>  
#include <fcntl.h>  
#include <sys/socket.h>  
#include <unistd.h>  
#include <cerrno>  
#include <cstring>  
#include <stdexcept>  
#include "RGBSlave.h"  
#include "packets.h"  
#include "wiringPi.h"
```

```
#include "wiringPiI2C.h"
```

Include dependency graph for BusServer.cpp:



Macros

- `#define RGB_SLAVE_ADDRESS 0x69`

7.24.1 Macro Definition Documentation

7.24.1.1 RGB_SLAVE_ADDRESS

```
#define RGB_SLAVE_ADDRESS 0x69
```

Definition at line 17 of file [BusServer.cpp](#).

7.25 BusServer.cpp

[Go to the documentation of this file.](#)

```

00001 #include "BusServer.h"
00002
00003 #include <arpa/inet.h>
00004 #include <fcntl.h>
00005 #include <sys/socket.h>
00006 #include <unistd.h>
00007
00008 #include <cerrno>
00009 #include <cstring>
00010 #include <stdexcept>
00011
00012 #include "RGBSlave.h"
00013 #include "packets.h"
00014 #include "wiringPi.h"
00015 #include "wiringPiI2C.h"
00016
00017 #define RGB_SLAVE_ADDRESS 0x69
00018
00019 void BusServer::setup(std::string ip, int port) {
00020     // slave_manager.initialize();
00021
00022     memset(&listening_address, 0, sizeof(listening_address));
00023
00024     if (port <= 0 || port > 65535) {
00025         throw std::invalid_argument("invalid port passed");
00026     }
00027
00028     if (0 == inet_aton(ip.c_str(), &listening_address.sin_addr)) {
00029         perror("inet_aton()");
00030         throw std::invalid_argument("invalid IP address passed");
00031     }
00032
00033     listening_address.sin_family = AF_INET;
00034     listening_address.sin_port = htons(port);

```

```

00035
00036     listening_fd = socket(AF_INET, SOCK_STREAM | SOCK_NONBLOCK, 0);
00037     if (-1 == listening_fd) {
00038         perror("socket()");
00039         throw std::runtime_error("failed to create socket");
00040     }
00041
00042     {
00043         int one = 1;
00044         if (-1 ==
00045             setsockopt(listening_fd, SOL_SOCKET, SO_REUSEADDR | SO_REUSEPORT, &one, sizeof(one))) {
00046             perror("setsockopt()");
00047             throw std::runtime_error("failed to set socket options");
00048         }
00049     }
00050
00051     socklen_t len = sizeof(listening_address);
00052
00053     if (-1 == bind(listening_fd, (struct sockaddr*)&listening_address, len)) {
00054         perror("bind()");
00055         throw std::runtime_error("failed to bind socket");
00056     }
00057 }
00058
00059 void BusServer::listen() {
00060     if (-1 == ::listen(listening_fd, 8)) {
00061         perror("listen()");
00062         throw std::runtime_error("failed to listen on socket");
00063     }
00064 }
00065
00066 void BusServer::send(struct sensor_packet* packet_ptr, int fd) {
00067     if (!wemos_bridge_connected) {
00068         throw std::runtime_error("no longer connected to socket");
00069     }
00070     if (-1 == ::send(fd, packet_ptr, sizeof(struct sensor_header) + packet_ptr->header.length, 0)) {
00071         perror("send()");
00072         throw std::runtime_error("failed to send on socket");
00073     }
00074 }
00075
00076 void BusServer::start() {
00077     // start off by mapping pre-known addresses to ID's
00078     listen();
00079
00080     int the_fd = -1;
00081
00082     while (true) {
00083         char buffer[32] = {0};
00084         bool net_request = false;
00085
00086         int new_fd = -1;
00087
00088         {
00089             struct sockaddr_in client_address = {0};
00090             socklen_t client_address_length = sizeof(client_address);
00091
00092             new_fd = accept4(listening_fd, (struct sockaddr*)&client_address,
00093                             &client_address_length, SOCK_NONBLOCK);
00094
00095             if (-1 == new_fd) {
00096                 int err = errno;
00097                 switch (err) {
00098                     case EWOULDBLOCK:
00099                         // no client has tried to connect; no big deal
00100                         break;
00101
00102                     default:
00103                         perror("accept4()");
00104                         throw std::runtime_error("accept4() failed");
00105                         break;
00106                 }
00107             } else {
00108                 the_fd = new_fd;
00109                 // printf("new fd: %d\nthe fd: %d\n", new_fd, the_fd);
00110             }
00111
00112             if (the_fd != -1) {
00113                 int err;
00114                 int recvd = recv(the_fd, buffer, sizeof(buffer), SOCK_NONBLOCK);
00115                 if (0 == recvd) {
00116                     close(the_fd);
00117                     the_fd = -1;
00118                     continue;
00119                 }
00120
00121                 if (-1 == recvd) err = errno;

```

```

00122         // printf("%d\n", err);
00123         // perror("recv balls");
00124         if (-1 == recvd && err != EAGAIN) {
00125             perror("recv()");
00126             usleep(100000);
00127             throw std::runtime_error("reading from client socket failed for some reason");
00128         } else if (errno != EAGAIN) {
00129             // printf("%d\n", recvd);
00130         } else {
00131             // printf("%d\n", recvd);
00132             if (recvd > 0) net_request = true;
00133
00134             // perror("recv() [1]");
00135             usleep(100000);
00136         }
00137     }
00138 }
00139
00140 if (net_request) {
00141     struct sensor_packet* pkt_ptr = (struct sensor_packet*)buffer;
00142     BaseSlave* the_slave = slave_manager.getSlave(pkt_ptr->data.generic.metadata.sensor_id);
00143     SensorType the_sensor_type = pkt_ptr->data.generic.metadata.sensor_type;
00144     uint8_t values[8] = {0};
00145
00146     switch (pkt_ptr->header.ptype) {
00147         case PacketType::DASHBOARD_POST:
00148             switch (the_sensor_type) {
00149                 case SensorType::LIGHT:
00150                     values[0] = pkt_ptr->data.light.target_state;
00151
00152                     the_slave->setData(values);
00153                     break;
00154
00155                 case SensorType::RGB_LIGHT:
00156                     struct RGBData rgb_data = {.R = pkt_ptr->data.rgb_light.red_state,
00157                                                 .G = pkt_ptr->data.rgb_light.green_state,
00158                                                 .B = pkt_ptr->data.rgb_light.blue_state};
00159
00160                     the_slave->setData(&rgb_data);
00161
00162                     break;
00163             }
00164             break;
00165
00166         case PacketType::DASHBOARD_GET:
00167             struct sensor_packet state_ptr;
00168             memcpy(&state_ptr, the_slave->getData(), sizeof(struct sensor_packet));
00169
00170             state_ptr.header.ptype = PacketType::DASHBOARD_RESPONSE;
00171
00172             send(&state_ptr, new_fd);
00173             break;
00174     }
00175 }
00176 }
00177 }
00178
00179 SlaveManager& BusServer::getSlaveManager() { return slave_manager; }

```

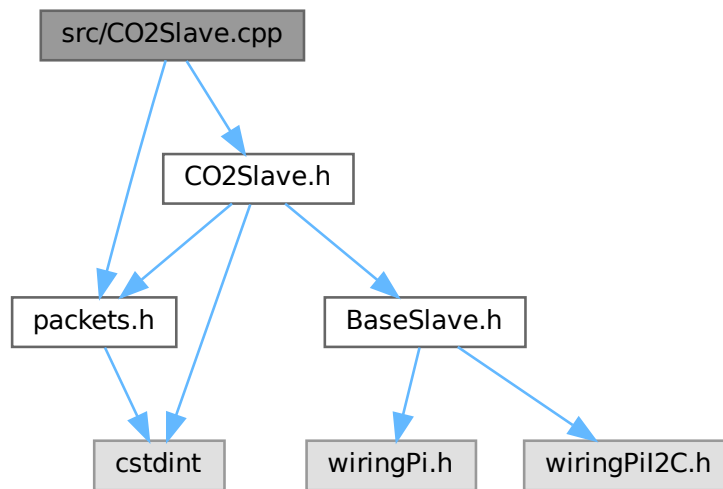
7.26 src/CO2Slave.cpp File Reference

```

#include "CO2Slave.h"
#include "packets.h"

```

Include dependency graph for CO2Slave.cpp:



7.27 CO2Slave.cpp

[Go to the documentation of this file.](#)

```

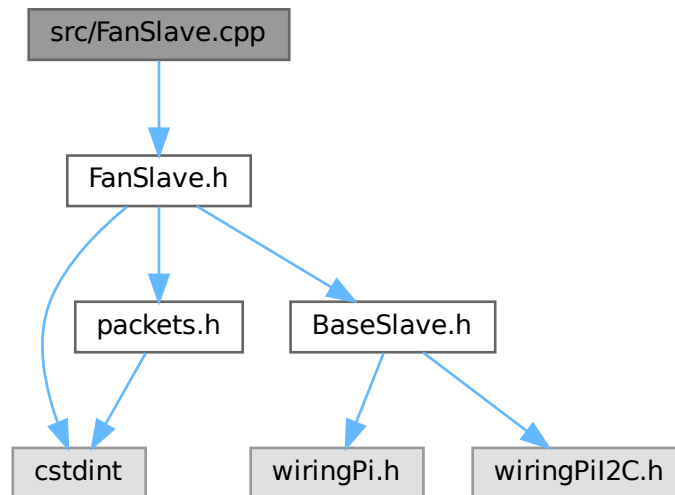
00001 #include "CO2Slave.h"
00002
00003 #include "packets.h"
00004
00005 CO2Slave::CO2Slave(uint8_t id, uint8_t i2c_address) : id(id) {
00006     state_packet.header.ptype = PacketType::DATA;
00007     state_packet.header.length = sizeof(struct sensor_packet_co2);
00008
00009     state_packet.data.rgb_light.metadata.sensor_id = id;
00010     state_packet.data.rgb_light.metadata.sensor_type = SensorType::CO2;
00011     command = 0x2003;
00012     uint8_t* command_ptr = (uint8_t*)&command;
00013
00014     wiringPiI2CWrite(fd, command_ptr[0]);
00015     wiringPiI2CWrite(fd, command_ptr[1]);
00016 }
00017
00018 const void* CO2Slave::getData() {
00019     command = 0x2008;
00020     uint8_t* command_ptr = (uint8_t*)&command;
00021     wiringPiI2CWrite(fd, command_ptr[0]);
00022     wiringPiI2CWrite(fd, command_ptr[1]);
00023     uint8_t* value_ptr = (uint8_t*)&state_packet.data.co2.value;
00024
00025     value_ptr[1] = wiringPiI2CRead(fd);
00026     value_ptr[0] = wiringPiI2CRead(fd);
00027     uint8_t empty = wiringPiI2CRead(fd);
00028     return &state_packet;
00029 }
00030
00031 bool CO2Slave::getStatus() { return power_state; }
00032
00033 int CO2Slave::getId() { return id; }
00034
00035 void CO2Slave::setData(void* data) { return; }
00036
00037 void CO2Slave::start(int i2c_fd) { fd = i2c_fd; }
00038
00039 void CO2Slave::stop() { fd = -1; }

```

7.28 src/FanSlave.cpp File Reference

```
#include "FanSlave.h"
```

Include dependency graph for FanSlave.cpp:



7.29 FanSlave.cpp

[Go to the documentation of this file.](#)

```

00001 #include "FanSlave.h"
00002
00003 FanSlave::FanSlave(uint8_t id, uint8_t i2c_address) : id(id), i2c_address(i2c_address) {}
00004
00005 const void* FanSlave::getData() { return &current_speed; }
00006
00007 bool FanSlave::getStatus() {
00008     if (current_speed == 0) {
00009         return false;
00010     } else {
00011         return true;
00012     }
00013 }
00014
00015 int FanSlave::getId() { return id; }
00016
00017 void FanSlave::setData(void* data) {
00018     current_speed = *(uint8_t*)data;
00019
00020     wiringPiI2CWrite(fd, current_speed);
00021 }
00022
00023 void FanSlave::start(int i2c_fd) { fd = i2c_fd; }
00024
00025 void FanSlave::stop() { fd = -1; }

```

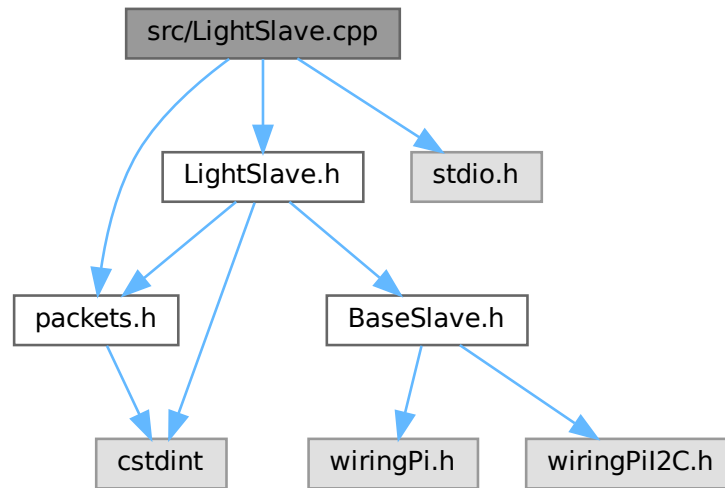
7.30 src/LightSlave.cpp File Reference

```
#include "LightSlave.h"
```

```
#include <stdio.h>
```

```
#include "packets.h"
```

Include dependency graph for LightSlave.cpp:



7.31 LightSlave.cpp

[Go to the documentation of this file.](#)

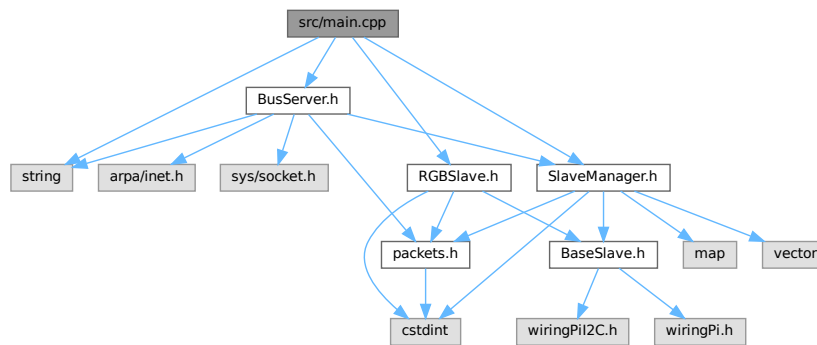
```

00001 #include "LightSlave.h"
00002
00003 #include <stdio.h>
00004
00005 #include "packets.h"
00006
00007 LightSlave::LightSlave(uint8_t id, uint8_t i2c_address) : id(id), i2c_address(i2c_address) {
00008     state_packet.header.ptype = PacketType::DATA;
00009     state_packet.header.length = sizeof(struct sensor_packet_light);
00010
00011     state_packet.data.light.metadata.sensor_id = id;
00012     state_packet.data.light.metadata.sensor_type = SensorType::LIGHT;
00013 }
00014
00015 const void* LightSlave::getData() {
00016     uint8_t value = wiringPiI2CRead(fd);
00017
00018     state_packet.data.light.target_state = value;
00019
00020     return &state_packet;
00021 }
00022
00023 bool LightSlave::getStatus() {
00024     getData();
00025     return state_packet.data.light.target_state;
00026 }
00027
00028 int LightSlave::getId() { return id; }
00029
00030 void LightSlave::setData(void* data) {
00031     uint8_t* value = (uint8_t*)data;
00032
00033     char command[256] = {0};
00034     snprintf(command, sizeof(command) - 1, "/usr/sbin/i2cset -y 1 0x%02x 0x%02x i", id, *value);
00035
00036     popen(command, "r");
00037 }
00038
00039 void LightSlave::start(int i2c_fd) { fd = i2c_fd; }
00040
00041 void LightSlave::stop() { fd = -1; }

```

7.32 src/main.cpp File Reference

```
#include <string>
#include "BusServer.h"
#include "RGBSlave.h"
#include "SlaveManager.h"
Include dependency graph for main.cpp:
```



Functions

- int [main](#) ()

7.32.1 Function Documentation

7.32.1.1 main()

```
int main ( )
```

Definition at line 7 of file [main.cpp](#).

7.33 main.cpp

[Go to the documentation of this file.](#)

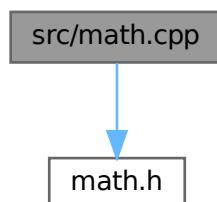
```
00001 #include <string>
00002
00003 #include "BusServer.h"
00004 #include "RGBSlave.h"
00005 #include "SlaveManager.h"
00006
00007 int main() {
00008     BusServer bus_server;
00009     bus_server.setup("0.0.0.0", 5000);
00010
00011     SlaveManager& slave_manager_ref = bus_server.getSlaveManager();
00012
00013     {
00014         int fd = wiringPiI2CSetup(0x69);
00015
00016         slave_manager_ref.createSlave(SensorType::RGB_LIGHT, 0x69);
00017         slave_manager_ref.getSlave(0x69)->start(fd);
00018     }
00019
00020     bus_server.start();
00021 }
```


7.34 src/math.cpp File Reference

Implementation of basic math operations.

```
#include "math.h"
```

Include dependency graph for math.cpp:



Functions

- int [add](#) (int a, int b)
Adds two integers.
- int [subtract](#) (int a, int b)
Subtracts two integers.

7.34.1 Detailed Description

Implementation of basic math operations.

Author

Daan Breur

Definition in file [math.cpp](#).

7.34.2 Function Documentation

7.34.2.1 add()

```
int add (  
    int a,  
    int b )
```

Adds two integers.

Parameters

<i>a</i>	First integer.
<i>b</i>	Second integer.

Returns

The sum of a and b.

This function takes two integers as input and returns their sum.

Definition at line 16 of file [math.cpp](#).

7.34.2.2 subtract()

```
int subtract (
    int a,
    int b )
```

Subtracts two integers.

Parameters

<i>a</i>	First integer.
<i>b</i>	Second integer.

Returns

The difference of a and b.

This function takes two integers as input and returns the result of subtracting b from a.

Definition at line 26 of file [math.cpp](#).

7.35 math.cpp

[Go to the documentation of this file.](#)

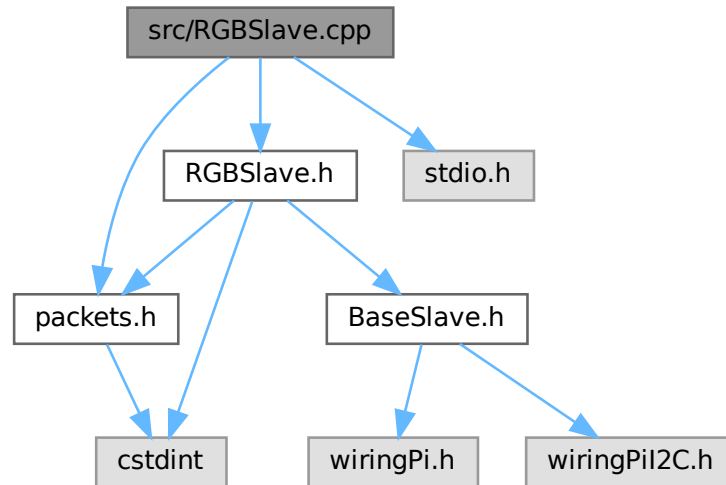
```
00001
00006 #include "math.h"
00007
00016 int add(int a, int b) { return a + b; }
00017
00026 int subtract(int a, int b) { return a - b; }
```

7.36 src/RGBSlave.cpp File Reference

```
#include "RGBSlave.h"
#include <stdio.h>
```

```
#include "packets.h"
```

Include dependency graph for RGBSlave.cpp:



7.37 RGBSlave.cpp

[Go to the documentation of this file.](#)

```

00001 #include "RGBSlave.h"
00002
00003 #include <stdio.h>
00004
00005 #include "packets.h"
00006
00007 RGBSlave::RGBSlave(uint8_t id, uint8_t i2c_address) : id(id), i2c_address(i2c_address) {
00008     state_packet.header.ptype = PacketType::DATA;
00009     state_packet.header.length = sizeof(struct sensor_packet_rgb_light);
00010
00011     state_packet.data.rgb_light.metadata.sensor_id = id;
00012     state_packet.data.rgb_light.metadata.sensor_type = SensorType::RGB_LIGHT;
00013 }
00014
00015 const void* RGBSlave::getData() {
00016     uint8_t r, g, b;
00017
00018     r = wiringPiI2CRead(fd);
00019     g = wiringPiI2CRead(fd);
00020     b = wiringPiI2CRead(fd);
00021
00022     color_state.R = r;
00023     color_state.G = g;
00024     color_state.B = b;
00025
00026     // state_packet.data.rgb_light.red_state = r;
00027     // state_packet.data.rgb_light.green_state = g;
00028     // state_packet.data.rgb_light.blue_state = b;
00029
00030     return &state_packet;
00031 }
00032
00033 bool RGBSlave::getStatus() {
00034     getData();
00035     if (color_state.R == 0 && color_state.G == 0 && color_state.B == 0) {
00036         return false;
00037     } else {
00038         return true;
00039     }
00040 }

```

```

00040 }
00041
00042 int RGBSlave::getId() { return id; }
00043
00044 void RGBSlave::setData(void* data) {
00045     RGBData* rgb_data = (RGBData*)data;
00046     uint8_t r = rgb_data->R;
00047     uint8_t g = rgb_data->G;
00048     uint8_t b = rgb_data->B;
00049
00050     char command[256] = {0};
00051     snprintf(command, sizeof(command) - 1, "/usr/sbin/i2cset -y 1 0x%02x 0x%02x 0x%02x 0x%02x i",
00052             id, r, g, b);
00053
00054     state_packet.data.rgb_light.red_state = r;
00055     state_packet.data.rgb_light.green_state = g;
00056     state_packet.data.rgb_light.blue_state = b;
00057
00058     popen(command, "r");
00059
00060     // wiringPiI2CWriteBlockData(fd, 1, (uint8_t*)rgb_data, 3);
00061     // wiringPiI2CWrite(fd, g);
00062     // wiringPiI2CWrite(fd, b);
00063 }
00064
00065 void RGBSlave::start(int i2c_fd) { fd = i2c_fd; }
00066
00067 void RGBSlave::stop() { fd = -1; }

```

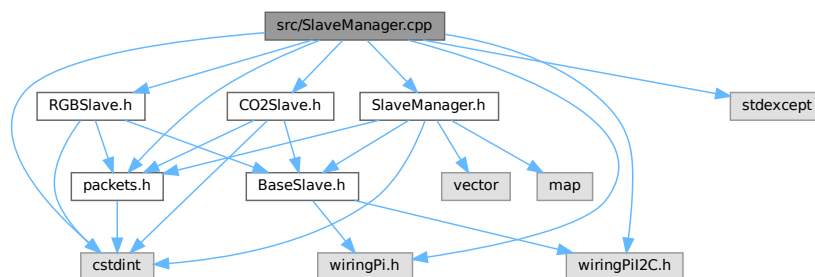
7.38 src/SlaveManager.cpp File Reference

```

#include "SlaveManager.h"
#include <wiringPi.h>
#include <wiringPiI2C.h>
#include <stdint>
#include <stdexcept>
#include "CO2Slave.h"
#include "RGBSlave.h"
#include "packets.h"

```

Include dependency graph for SlaveManager.cpp:



Macros

- `#define MASTER_ADDRESS 0x01`

7.38.1 Macro Definition Documentation

7.38.1.1 MASTER_ADDRESS

```
#define MASTER_ADDRESS 0x01
```

Definition at line 13 of file [SlaveManager.cpp](#).

7.39 SlaveManager.cpp

[Go to the documentation of this file.](#)

```

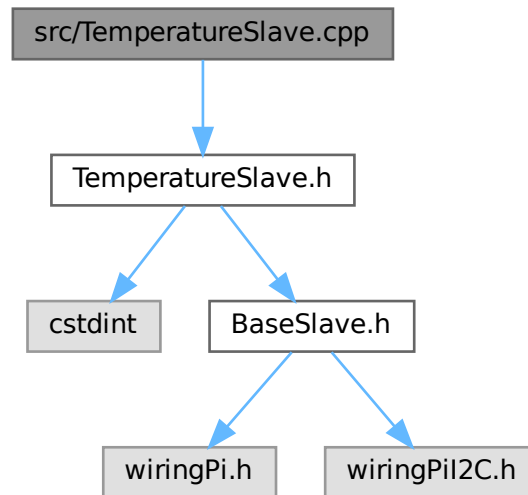
00001 #include "SlaveManager.h"
00002
00003 #include <wiringPi.h>
00004 #include <wiringPiI2C.h>
00005
00006 #include <cstdint>
00007 #include <stdexcept>
00008
00009 #include "CO2Slave.h"
00010 #include "RGBSlave.h"
00011 #include "packets.h"
00012
00013 #define MASTER_ADDRESS 0x01
00014
00015 SlaveManager::SlaveManager() : address(MASTER_ADDRESS) {}
00016
00017 void SlaveManager::initialize() {
00018     // maybe need setup not sure.
00019
00020     i2c_fd = wiringPiI2CSetup(address);
00021     if (-1 == i2c_fd) {
00022         throw std::runtime_error("I2C setup failed");
00023     } else {
00024         printf("I2C setup succesful");
00025     };
00026 }
00027
00028 void SlaveManager::ledControl(uint8_t led_number, uint8_t led_state) {}
00029
00030 void SlaveManager::createSlave(SensorType type, int i2c_address) {
00031     BaseSlave* newSlave = nullptr;
00032     switch (type) {
00033         case SensorType::CO2:
00034             newSlave = new CO2Slave(i2c_address, i2c_address);
00035             break;
00036         case SensorType::RGB_LIGHT:
00037             newSlave = new RGBSlave(i2c_address, i2c_address);
00038             break;
00039         // case /* door */:
00040         //     newSlave = new DoorSlave(id, i2c_address);
00041         //     break;
00042         default:
00043             return; // Invalid type
00044     }
00045     slaves[i2c_address] = newSlave;
00046 }
00047 void SlaveManager::deleteSlave(uint8_t id) {
00048     if (nullptr != slaves[id]) {
00049         /*
00050          * Disconnect I2C device from bus
00051          */
00052         delete slaves[id];
00053     }
00054 }
00055
00056 BaseSlave* SlaveManager::getSlave(int id) { return slaves[id]; }

```

7.40 src/TemperatureSlave.cpp File Reference

```
#include "TemperatureSlave.h"
```

Include dependency graph for TemperatureSlave.cpp:



7.41 TemperatureSlave.cpp

[Go to the documentation of this file.](#)

```

00001 #include "TemperatureSlave.h"
00002
00003 TemperatureSlave::TemperatureSlave(uint8_t id, uint8_t i2c_address)
00004     : id(id), i2c_address(i2c_address) {}
00005
00006 void* TemperatureSlave::getData() {
00007     int temp;
00008
00009     temp = wiringPiI2CRead(fd);
00010
00011     return;
00012 }
00013
00014 bool TemperatureSlave::getStatus() {
00015     getData();
00016     if (temperature.Temp == 0) {
00017         return false;
00018     } else {
00019         return true;
00020     }
00021 }
00022
00023 int TemperatureSlave::getId() { return id; }
00024
00025 void TemperatureSlave::setData(void* data) {
00026     TemperatureData* temp_data = (TemperatureData*)data;
00027     int temp = temp_data->Temp;
00028
00029     wiringPiI2CWrite(fd, temp);
00030 }
00031
00032 void TemperatureSlave::start(int i2c_fd) { fd = i2c_fd; }
00033
00034 void TemperatureSlave::stop() { fd = -1; }

```

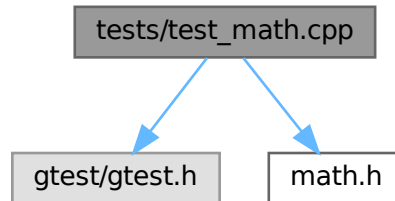
7.42 tests/test_math.cpp File Reference

Unit tests for mathematical operations using Google Test framework.

```
#include <gtest/gtest.h>
```

```
#include "math.h"
```

Include dependency graph for test_math.cpp:



Functions

- [TEST](#) (MathTest, Add)
- [TEST](#) (MathTest, Subtract)
- [TEST](#) (MathTest, SubtractNegative)

7.42.1 Detailed Description

Unit tests for mathematical operations using Google Test framework.

This file contains test cases for verifying the correctness of functions defined in the "math.h" header. The tests ensure that the mathematical operations behave as expected under various conditions.

[Test](#) MathTest.Add

- Verifies that the `add` function correctly computes the sum of two integers.
- Example: `add(2, 3)` should return 5.

[Test](#) MathTest.Subtract

- Verifies that the `subtract` function correctly computes the difference between two integers.
- Examples:
 - `subtract(10, 3)` should return 7.
 - `subtract(9, 3)` should return 6.

[Test](#) MathTest.SubtractNegative

- Verifies that the `subtract` function handles subtraction with negative integers correctly.
- Example: `subtract(10, -3)` should return 13.

Definition in file [test_math.cpp](#).

7.42.2 Function Documentation

7.42.2.1 TEST() [1/3]

```
TEST (
    MathTest ,
    Add )
```

Definition at line 29 of file [test_math.cpp](#).

7.42.2.2 TEST() [2/3]

```
TEST (
    MathTest ,
    Subtract )
```

Definition at line 31 of file [test_math.cpp](#).

7.42.2.3 TEST() [3/3]

```
TEST (
    MathTest ,
    SubtractNegative )
```

Definition at line 36 of file [test_math.cpp](#).

7.43 test_math.cpp

[Go to the documentation of this file.](#)

```
00001
00025 #include <gtest/gtest.h>
00026
00027 #include "math.h"
00028
00029 TEST(MathTest, Add) { EXPECT_EQ(add(2, 3), 5); }
00030
00031 TEST(MathTest, Subtract) {
00032     EXPECT_EQ(subtract(10, 3), 7);
00033     EXPECT_EQ(subtract(9, 3), 6);
00034 }
00035
00036 TEST(MathTest, SubtractNegative) { EXPECT_EQ(subtract(10, -3), 13); }
```


Index

- `__attribute__`
 - `packets.h`, [77](#)
 - `RGBSlave.h`, [82](#)
 - `TemperatureSlave.h`, [86](#)
 - `~BaseSlave`
 - `BaseSlave`, [12](#)
- `add`
 - `math.cpp`, [95](#)
 - `math.h`, [73](#)
- `address`
 - `SlaveManager`, [57](#)
- B**
 - `RGBData`, [31](#)
 - `RGBSlave.h`, [82](#)
- `BaseSlave`, [11](#)
 - `~BaseSlave`, [12](#)
 - `getData`, [12](#)
 - `getId`, [12](#)
 - `getStatus`, [13](#)
 - `setData`, [13](#)
 - `start`, [13](#)
 - `stop`, [13](#)
- `blue_state`
 - `packets.h`, [77](#)
 - `sensor_packet_rgb_light`, [53](#)
- `buffer`
 - `BusServer`, [16](#)
- `BUFFER_SIZE`
 - `BusServer.h`, [67](#)
- `Bus Bridge Server`, [1](#)
- `BusServer`, [14](#)
 - `buffer`, [16](#)
 - `BusServer`, [15](#)
 - `getSlaveManager`, [15](#)
 - `listen`, [15](#)
 - `listening_address`, [16](#)
 - `listening_fd`, [17](#)
 - `send`, [15](#)
 - `setup`, [16](#)
 - `slave_manager`, [17](#)
 - `start`, [16](#)
 - `wemos_bridge_connected`, [17](#)
- `BusServer.cpp`
 - `RGB_SLAVE_ADDRESS`, [88](#)
- `BusServer.h`
 - `BUFFER_SIZE`, [67](#)
- `BUTTON`
 - `packets.h`, [77](#)

- `CO2`
 - `packets.h`, [77](#)
- `co2`
 - `sensor_data`, [37](#)
 - `sensor_packet::sensor_data`, [38](#)
- `CO2Slave`, [17](#)
 - `CO2Slave`, [20](#)
 - `command`, [21](#)
 - `fd`, [21](#)
 - `getData`, [20](#)
 - `getId`, [20](#)
 - `getStatus`, [20](#)
 - `i2c_address`, [21](#)
 - `id`, [21](#)
 - `power_state`, [21](#)
 - `setData`, [20](#)
 - `start`, [20](#)
 - `state_packet`, [21](#)
 - `stop`, [21](#)
- `color_state`
 - `LightSlave`, [29](#)
 - `RGBSlave`, [35](#)
- `command`
 - `CO2Slave`, [21](#)
- `createSlave`
 - `SlaveManager`, [56](#)
- `current_speed`
 - `FanSlave`, [25](#)
- `DASHBOARD_GET`
 - `packets.h`, [77](#)
- `DASHBOARD_POST`
 - `packets.h`, [77](#)
- `DASHBOARD_RESPONSE`
 - `packets.h`, [77](#)
- `DATA`
 - `packets.h`, [77](#)
- `data`
 - `packets.h`, [77](#)
 - `sensor_packet`, [45](#)
- `deleteSlave`
 - `SlaveManager`, [56](#)
- `FAN`
 - `packets.h`, [77](#)
- `FanData`
 - `FanSlave.h`, [70](#)
- `FanSlave`, [22](#)
 - `current_speed`, [25](#)
 - `FanSlave`, [24](#)

- fd, 25
- getData, 24
- getId, 24
- getStatus, 24
- i2c_address, 25
- id, 25
- setData, 24
- start, 24
- state_packet, 25
- stop, 25
- FanSlave.h
 - FanData, 70
- fd
 - CO2Slave, 21
 - FanSlave, 25
 - LightSlave, 29
 - RGBSlave, 35
 - TemperatureSlave, 63
- G
 - RGBData, 31
 - RGBSlave.h, 82
- generic
 - sensor_data, 37
 - sensor_packet::sensor_data, 38
- getData
 - BaseSlave, 12
 - CO2Slave, 20
 - FanSlave, 24
 - LightSlave, 28
 - RGBSlave, 34
 - TemperatureSlave, 62
- getId
 - BaseSlave, 12
 - CO2Slave, 20
 - FanSlave, 24
 - LightSlave, 28
 - RGBSlave, 34
 - TemperatureSlave, 62
- getSlave
 - SlaveManager, 57
- getSlaveManager
 - BusServer, 15
- getStatus
 - BaseSlave, 13
 - CO2Slave, 20
 - FanSlave, 24
 - LightSlave, 28
 - RGBSlave, 34
 - TemperatureSlave, 62
- green_state
 - packets.h, 78
 - sensor_packet_rgb_light, 53
- header
 - packets.h, 78
 - sensor_packet, 45
- HEARTBEAT
 - packets.h, 77
- heartbeat
 - sensor_data, 37
 - sensor_packet::sensor_data, 38
- HUMIDITY
 - packets.h, 77
- humidity
 - sensor_data, 37
 - sensor_packet::sensor_data, 39
- i2c_address
 - CO2Slave, 21
 - FanSlave, 25
 - LightSlave, 29
 - RGBSlave, 35
 - TemperatureSlave, 63
- i2c_fd
 - SlaveManager, 57
- id
 - CO2Slave, 21
 - FanSlave, 25
 - LightSlave, 29
 - RGBSlave, 35
 - TemperatureSlave, 63
- include/BaseSlave.h, 65, 66
- include/BusServer.h, 66, 67
- include/CO2Slave.h, 68
- include/FanSlave.h, 69, 70
- include/LightSlave.h, 71, 72
- include/math.h, 72, 74
- include/packets.h, 74, 80
- include/RGBSlave.h, 81, 83
- include/SlaveManager.h, 83, 84
- include/TemperatureSlave.h, 85, 86
- initialize
 - SlaveManager, 57
- initialized
 - SlaveManager, 58
- ledControl
 - SlaveManager, 57
- length
 - packets.h, 78
 - sensor_header, 40
- LIGHT
 - packets.h, 77
- light
 - sensor_data, 37
 - sensor_packet::sensor_data, 39
- LightData
 - LightSlave.h, 72
- LightSlave, 26
 - color_state, 29
 - fd, 29
 - getData, 28
 - getId, 28
 - getStatus, 28
 - i2c_address, 29
 - id, 29
 - LightSlave, 28

- setData, 28
- start, 28
- state_packet, 29
- stop, 29
- LightSlave.h
 - LightData, 72
- listen
 - BusServer, 15
- listening_address
 - BusServer, 16
- listening_fd
 - BusServer, 17
- main
 - main.cpp, 94
- main.cpp
 - main, 94
- MASTER_ADDRESS
 - SlaveManager.cpp, 98
- math.cpp
 - add, 95
 - subtract, 96
- math.h
 - add, 73
 - subtract, 73
- metadata
 - packets.h, 78
 - sensor_heartbeat, 42
 - sensor_packet_co2, 46
 - sensor_packet_fan, 48
 - sensor_packet_generic, 49
 - sensor_packet_humidity, 50
 - sensor_packet_light, 52
 - sensor_packet_rgb_light, 53
 - sensor_packet_temperature, 55
- MOTION
 - packets.h, 77
- NOOP
 - packets.h, 77
- packets.h
 - __attribute__, 77
 - blue_state, 77
 - BUTTON, 77
 - CO2, 77
 - DASHBOARD_GET, 77
 - DASHBOARD_POST, 77
 - DASHBOARD_RESPONSE, 77
 - DATA, 77
 - data, 77
 - FAN, 77
 - green_state, 78
 - header, 78
 - HEARTBEAT, 77
 - HUMIDITY, 77
 - length, 78
 - LIGHT, 77
 - metadata, 78
 - MOTION, 77
 - NOOP, 77
 - PacketType, 76
 - PRESSURE, 77
 - ptype, 78
 - red_state, 78
 - RGB_LIGHT, 77
 - sensor_id, 78
 - sensor_type, 79
 - SensorType, 77
 - speed, 79
 - target_state, 79
 - TEMPERATURE, 77
 - value, 79
- PacketType
 - packets.h, 76
- power_state
 - CO2Slave, 21
 - TemperatureSlave, 63
- PRESSURE
 - packets.h, 77
- ptype
 - packets.h, 78
 - sensor_header, 40
- R
 - RGBData, 31
 - RGBSlave.h, 82
- README.md, 87
- red_state
 - packets.h, 78
 - sensor_packet_rgb_light, 53
- RGB_LIGHT
 - packets.h, 77
- rgb_light
 - sensor_data, 37
 - sensor_packet::sensor_data, 39
- RGB_SLAVE_ADDRESS
 - BusServer.cpp, 88
- RGBData, 30
 - B, 31
 - G, 31
 - R, 31
- RGBSlave, 31
 - color_state, 35
 - fd, 35
 - getData, 34
 - getId, 34
 - getStatus, 34
 - i2c_address, 35
 - id, 35
 - RGBSlave, 34
 - setData, 34
 - start, 34
 - state_packet, 35
 - stop, 35
- RGBSlave.h
 - __attribute__, 82
 - B, 82

- G, [82](#)
- R, [82](#)
- send
 - BusServer, [15](#)
- sensor_data, [36](#)
 - co2, [37](#)
 - generic, [37](#)
 - heartbeat, [37](#)
 - humidity, [37](#)
 - light, [37](#)
 - rgb_light, [37](#)
 - temperature, [37](#)
- sensor_header, [40](#)
 - length, [40](#)
 - ptype, [40](#)
- sensor_heartbeat, [41](#)
 - metadata, [42](#)
- sensor_id
 - packets.h, [78](#)
 - sensor_metadata, [43](#)
- sensor_metadata, [42](#)
 - sensor_id, [43](#)
 - sensor_type, [43](#)
- sensor_packet, [43](#)
 - data, [45](#)
 - header, [45](#)
- sensor_packet::sensor_data, [38](#)
 - co2, [38](#)
 - generic, [38](#)
 - heartbeat, [38](#)
 - humidity, [39](#)
 - light, [39](#)
 - rgb_light, [39](#)
 - temperature, [39](#)
- sensor_packet_co2, [45](#)
 - metadata, [46](#)
 - value, [46](#)
- sensor_packet_fan, [47](#)
 - metadata, [48](#)
 - speed, [48](#)
- sensor_packet_generic, [48](#)
 - metadata, [49](#)
- sensor_packet_humidity, [49](#)
 - metadata, [50](#)
 - value, [50](#)
- sensor_packet_light, [51](#)
 - metadata, [52](#)
 - target_state, [52](#)
- sensor_packet_rgb_light, [52](#)
 - blue_state, [53](#)
 - green_state, [53](#)
 - metadata, [53](#)
 - red_state, [53](#)
- sensor_packet_temperature, [54](#)
 - metadata, [55](#)
 - value, [55](#)
- sensor_type
 - packets.h, [79](#)
 - sensor_metadata, [43](#)
- SensorType
 - packets.h, [77](#)
- setData
 - BaseSlave, [13](#)
 - CO2Slave, [20](#)
 - FanSlave, [24](#)
 - LightSlave, [28](#)
 - RGBSlave, [34](#)
 - TemperatureSlave, [62](#)
- setup
 - BusServer, [16](#)
- slave_manager
 - BusServer, [17](#)
- SlaveManager, [55](#)
 - address, [57](#)
 - createSlave, [56](#)
 - deleteSlave, [56](#)
 - getSlave, [57](#)
 - i2c_fd, [57](#)
 - initialize, [57](#)
 - initialized, [58](#)
 - ledControl, [57](#)
 - SlaveManager, [56](#)
 - slaves, [58](#)
- SlaveManager.cpp
 - MASTER_ADDRESS, [98](#)
- slaves
 - SlaveManager, [58](#)
- speed
 - packets.h, [79](#)
 - sensor_packet_fan, [48](#)
- src/BaseSlave.cpp, [87](#)
- src/BusServer.cpp, [87](#), [88](#)
- src/CO2Slave.cpp, [90](#), [91](#)
- src/FanSlave.cpp, [92](#)
- src/LightSlave.cpp, [92](#), [93](#)
- src/main.cpp, [94](#)
- src/math.cpp, [95](#), [96](#)
- src/RGBSlave.cpp, [96](#), [97](#)
- src/SlaveManager.cpp, [98](#), [99](#)
- src/TemperatureSlave.cpp, [100](#)
- start
 - BaseSlave, [13](#)
 - BusServer, [16](#)
 - CO2Slave, [20](#)
 - FanSlave, [24](#)
 - LightSlave, [28](#)
 - RGBSlave, [34](#)
 - TemperatureSlave, [62](#)
- state_packet
 - CO2Slave, [21](#)
 - FanSlave, [25](#)
 - LightSlave, [29](#)
 - RGBSlave, [35](#)
- stop
 - BaseSlave, [13](#)
 - CO2Slave, [21](#)

- FanSlave, [25](#)
- LightSlave, [29](#)
- RGBSlave, [35](#)
- TemperatureSlave, [63](#)
- subtract
 - math.cpp, [96](#)
 - math.h, [73](#)
- target_state
 - packets.h, [79](#)
 - sensor_packet_light, [52](#)
- Temp
 - TemperatureData, [59](#)
 - TemperatureSlave.h, [86](#)
- TEMPERATURE
 - packets.h, [77](#)
- temperature
 - sensor_data, [37](#)
 - sensor_packet::sensor_data, [39](#)
 - TemperatureSlave, [63](#)
- TemperatureData, [58](#)
 - Temp, [59](#)
- TemperatureSlave, [59](#)
 - fd, [63](#)
 - getData, [62](#)
 - getId, [62](#)
 - getStatus, [62](#)
 - i2c_address, [63](#)
 - id, [63](#)
 - power_state, [63](#)
 - setData, [62](#)
 - start, [62](#)
 - stop, [63](#)
 - temperature, [63](#)
 - TemperatureSlave, [62](#)
- TemperatureSlave.h
 - __attribute__, [86](#)
 - Temp, [86](#)
- TEST
 - test_math.cpp, [102](#)
- Test List, [3](#)
- test_math.cpp
 - TEST, [102](#)
- tests/test_math.cpp, [101](#), [102](#)
- value
 - packets.h, [79](#)
 - sensor_packet_co2, [46](#)
 - sensor_packet_humidity, [50](#)
 - sensor_packet_temperature, [55](#)
- wemos_bridge_connected
 - BusServer, [17](#)